RDFIA · DEEP LEARNING FOR VISION

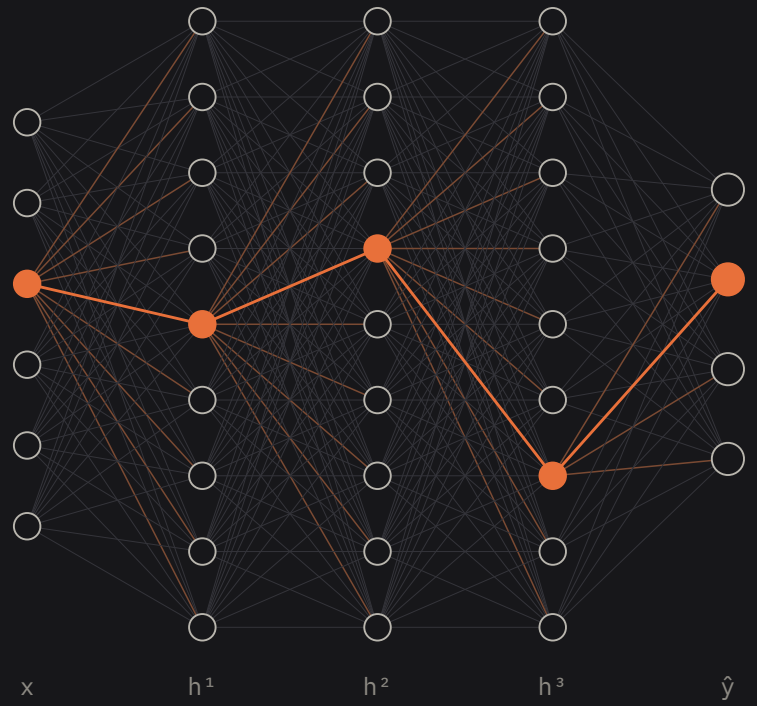
Large ConvNets & Vision Transformers

Clément Rambour

Sorbonne University

clement.rambour@sorbonne-universite.fr

clementrambour.github.io/teaching-rdfia



PART 1 / 3

Large ConvNets

01 **Large ConvNets**

02 Vision transformers

03 Deep residual learning

Recap AlexNet: what's next?

- How to improve AlexNet architecture?
- +++Deep?
- +++Convolutional?
- +++Fully connected?
- All?
- A lot of empirical studies
  - Tuning various design parameters
  - what really works?
- Winners: GoogLeNet, VGG, ResNet

LARGE CONVNETS · WINNERS

How to improve AlexNet?

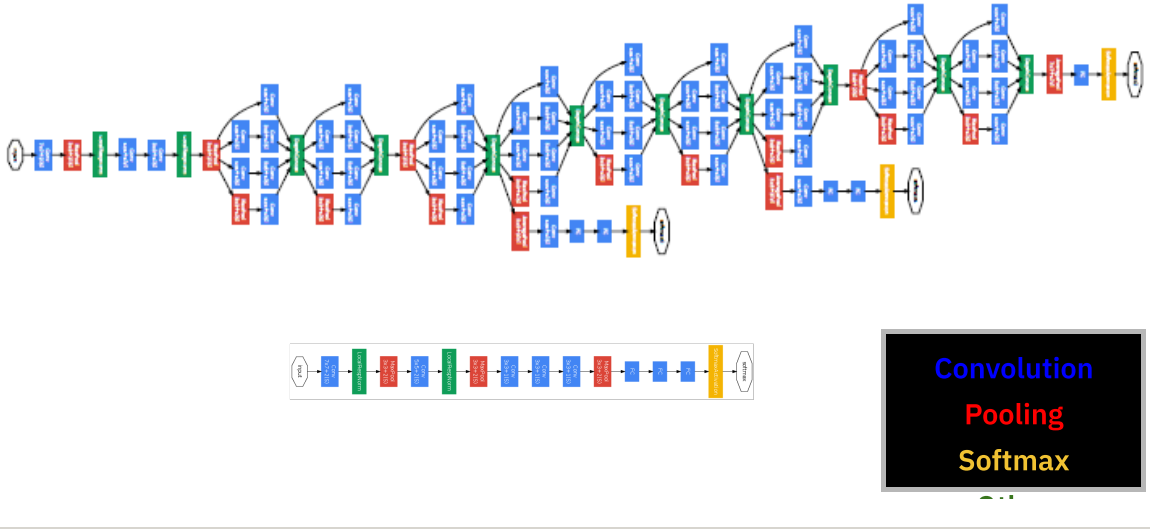
01 **GoogLeNet**

02 VGG

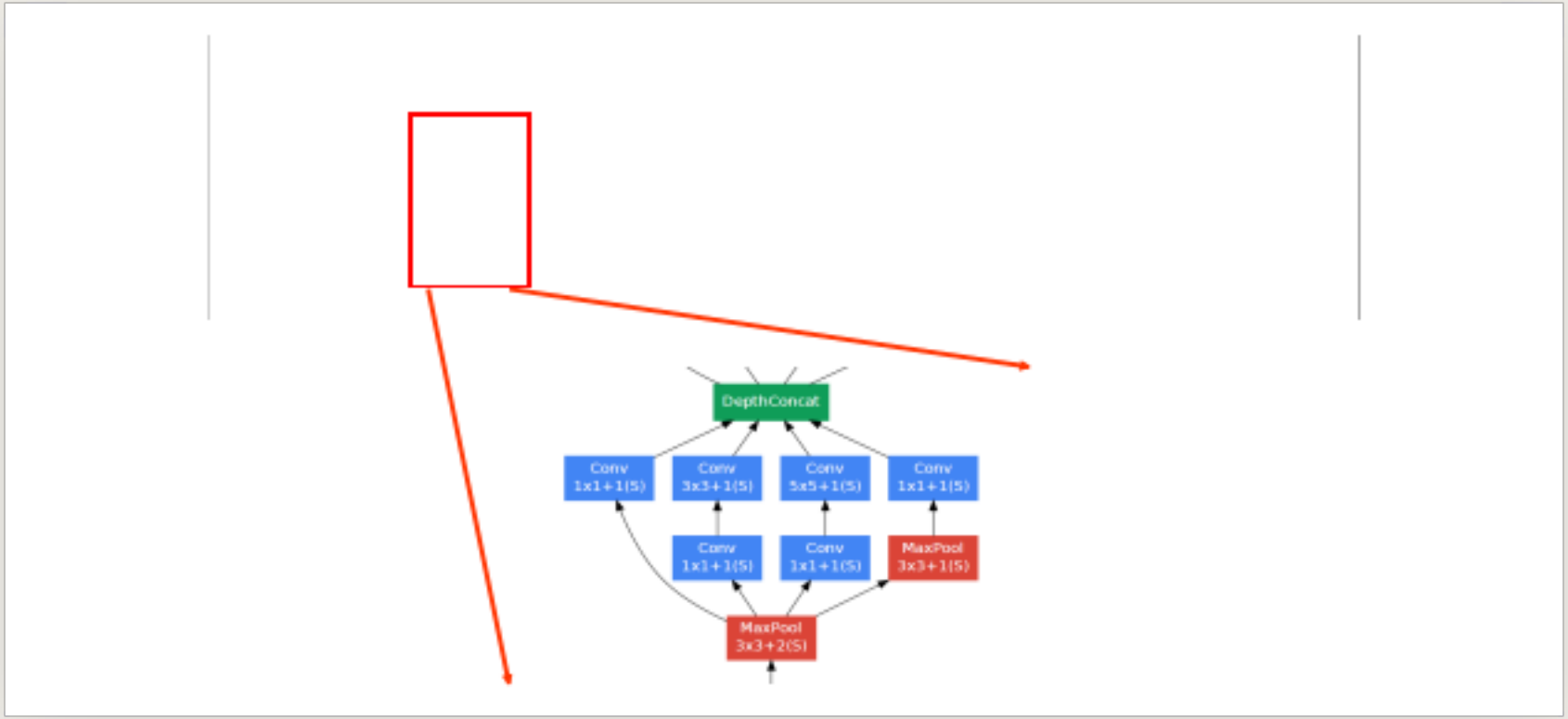
03 ResNet

GoogLeNet (2014)

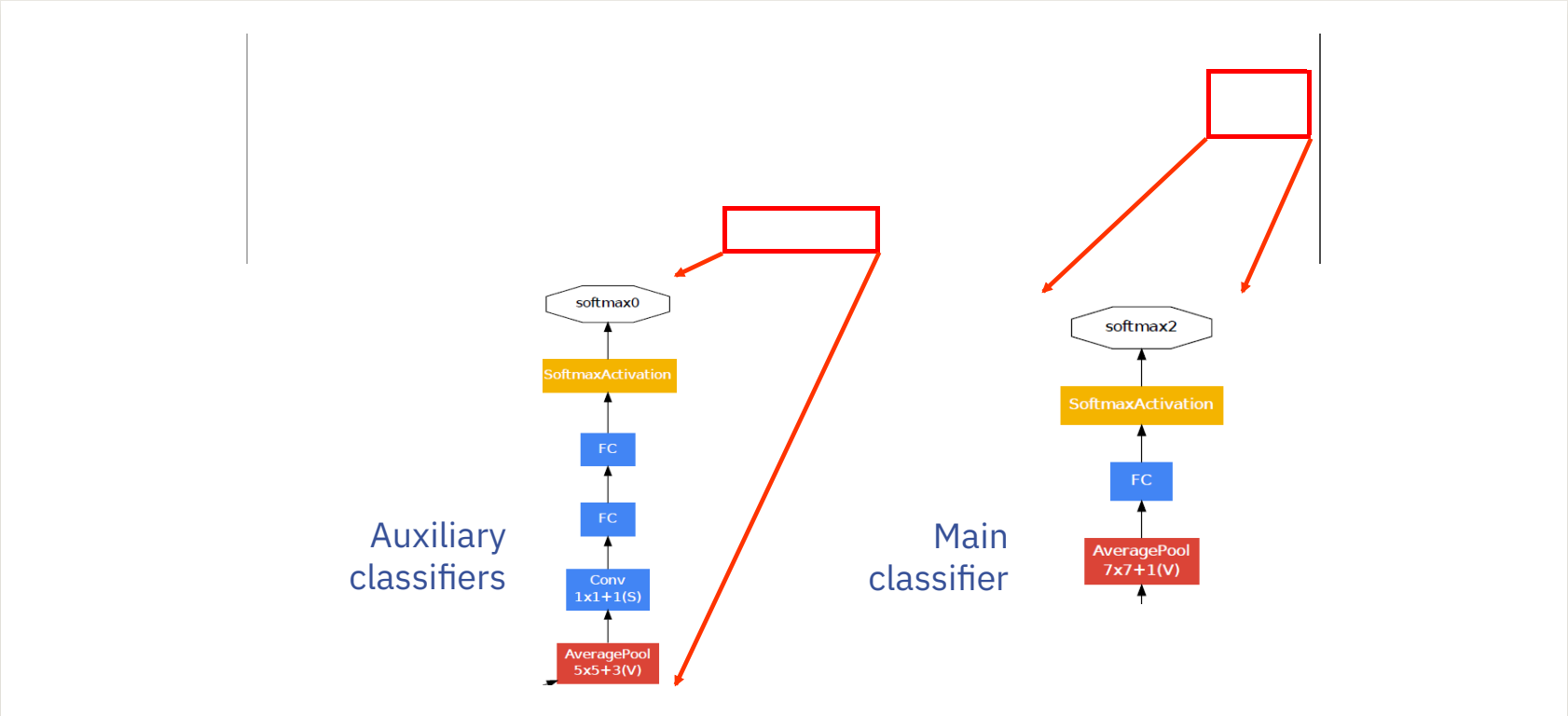
- Winner of ILSVRC -2014. Very deep network with 22 layers:
- Network–in-network–in-network
- Removed fully connected layers → small # of parameters (5M weights)



GoogLeNet: the Inception layer



GoogLeNet: auxiliary classifiers



LARGE CONVNETS · WINNERS

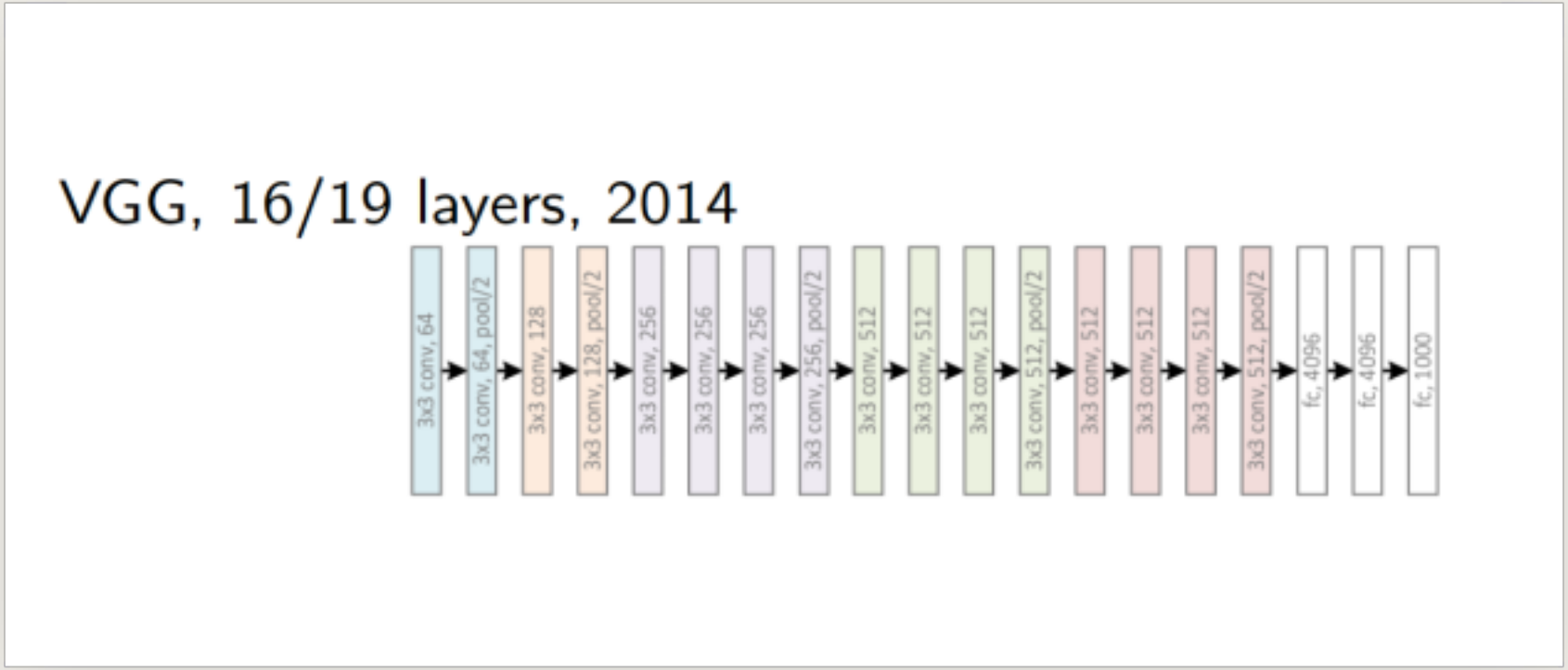
How to improve AlexNet?

01 GoogLeNet

02 **VGG**

03 ResNet

VGG Net: architecture post-2012 revolution



K. Simonyan, A. Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition. ICLR 2015

VGG Net: basic idea

- Basic Idea: Investigate the **effect of depth** in large scale image recognition
- **Fix other parameters** of architecture, and steadily increase depth

VGG Net: fixed configuration

- Convolutional Layers: from 8 to 16
- Fully Connected Layers: 3
- Stride: 1
- ReLu: Follow all hidden layers
- Max-Pooling: 2x2 window
- Padding: s/t spatial resolution is preserved
- #Convolutional filters: Starting from 64, double after each max-pooling layer until 512
- Filter sizes: 3x3 and 1x1

VGG Net: configurations

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table credit: Very Deep Convolutional Networks for Large-Scale Image Recognition, ICLR 2015

VGG Net: results

- Results:
- First place in localization (25.3% error), second in classification (7.3% error) in ILSVRC 2014 using ensemble of 7 networks
- Outperforms Szegedy et.al (GoogLeNet) in terms of single network classification accuracy (7.1% vs 7.9%)

Observations with VGG testing

- Deepnets with small filters outperform shallow networks with large filters
  - Shallow version of B: 2 layers of 3x3 replaced with single 5x5 performs worse
- Classification error decreases with increases ConvNet depth
- Important to capture more spatial context (config D vs C)
- Error rate saturated at 19 layers
- Scale jittering at training helps capturing multiscale statistics and leads to better performance

LARGE CONVNETS · WINNERS

How to improve AlexNet?

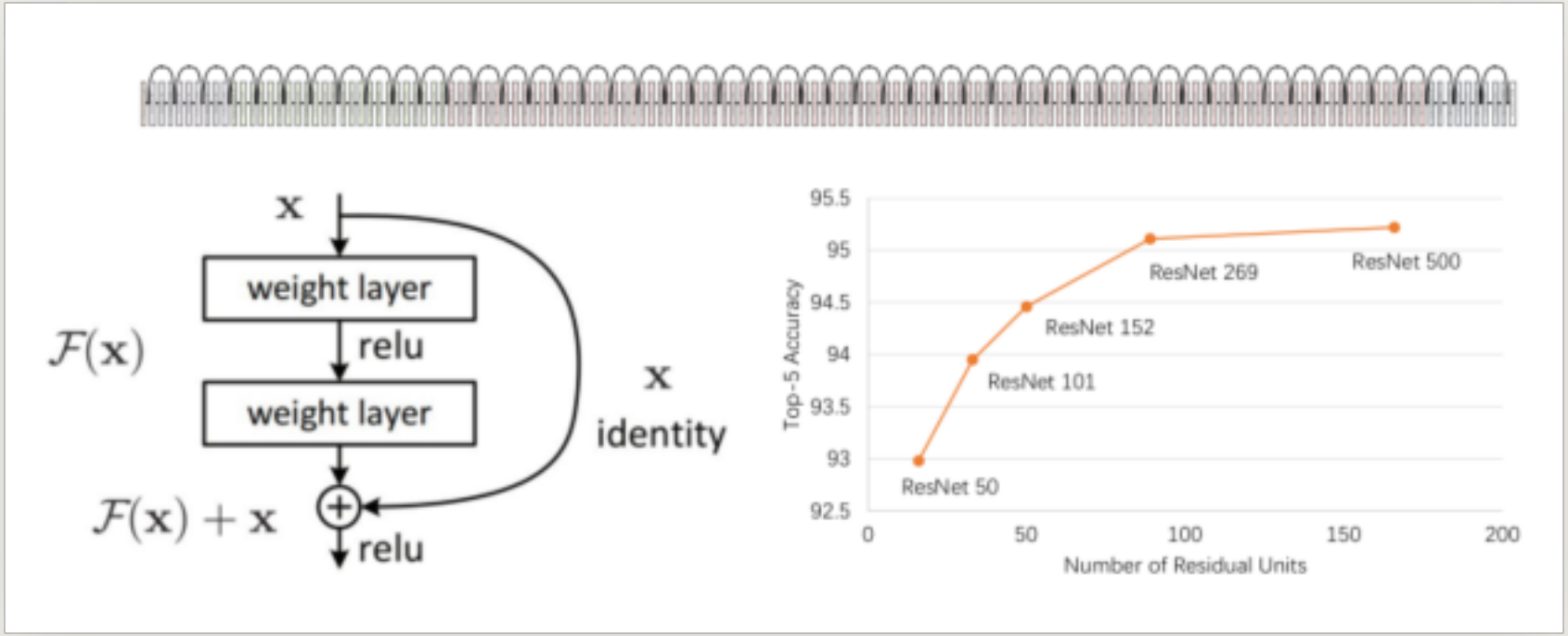
01 GoogLeNet

02 VGG

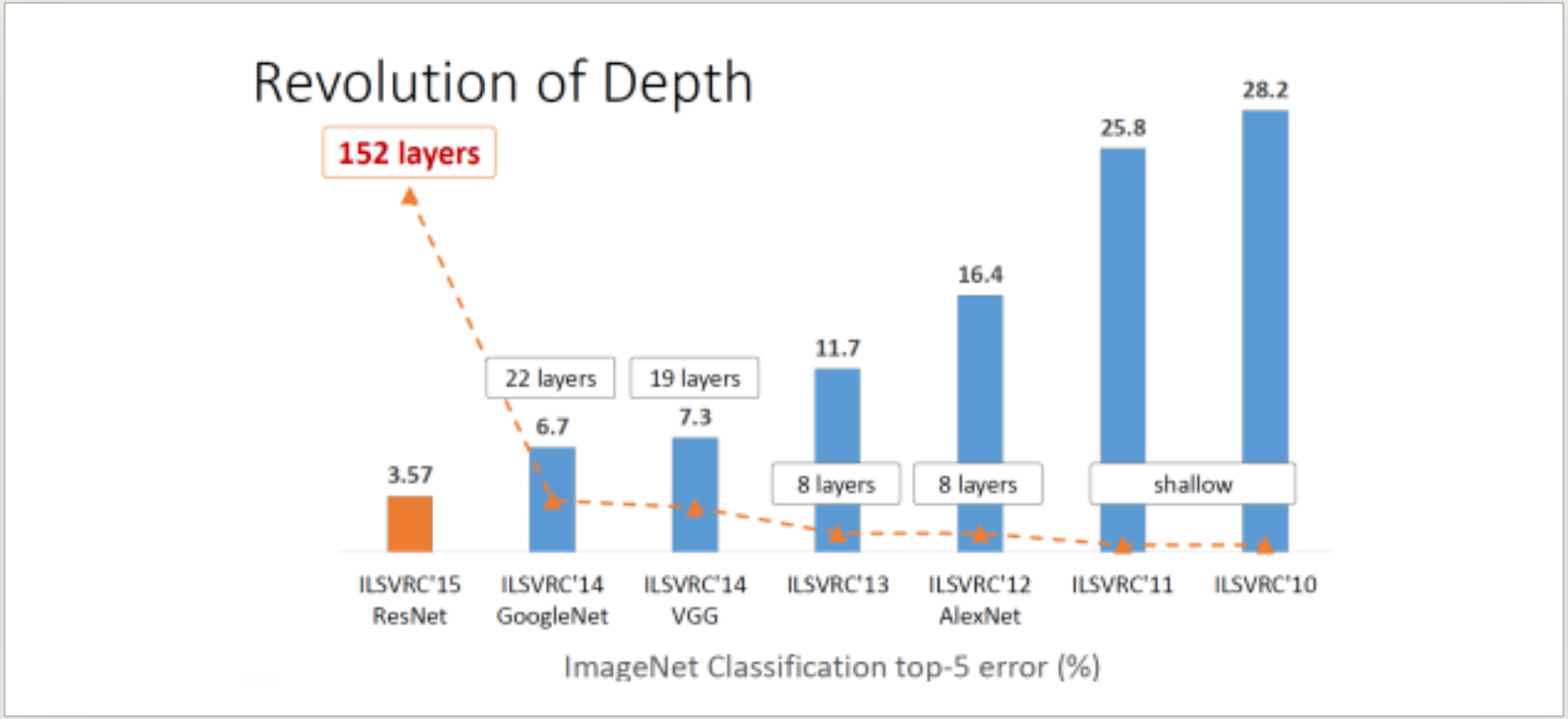
03 **ResNet**

Deep ConvNets for image classification

- ResNet: 152 layers, 60M parameters



Deep ConvNets for image classification



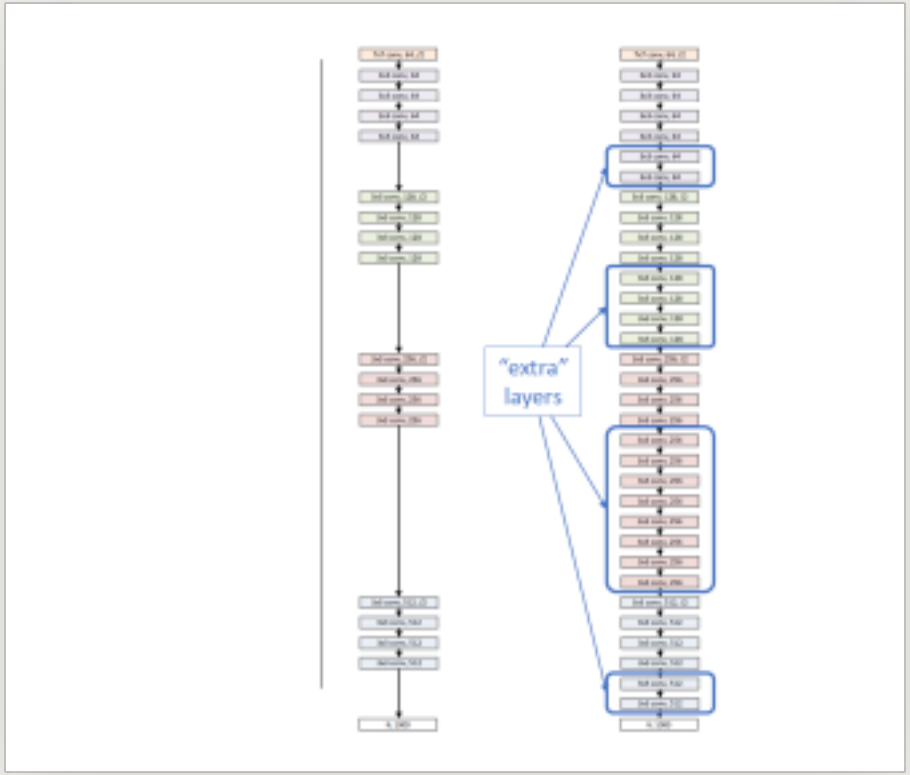
ResNet: the deeper, the better

- + Deeper network covers more complex problems
  - Receptive field size ↑
  - Non-linearity ↑
- Training deeper network more difficult because of vanishing/exploding gradients problem

© Kaiming He, ILSVRC & COCO 2015

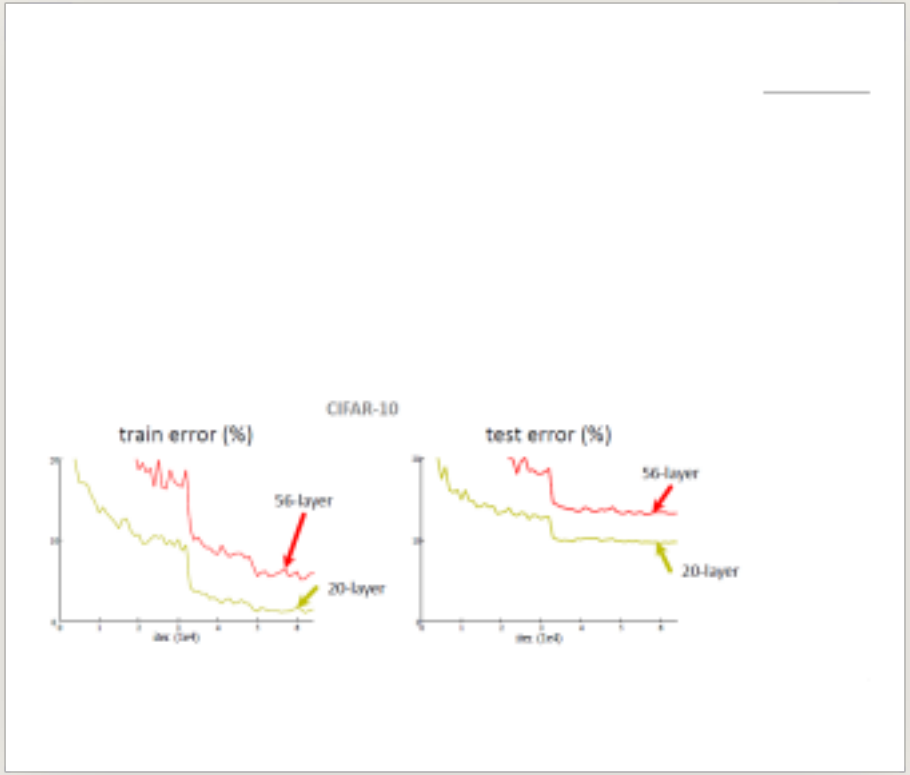
Deeper VGG: naïve solution

- Naïve solution
  - If the extra layers are identity mappings, the deeper network should not have a higher training error



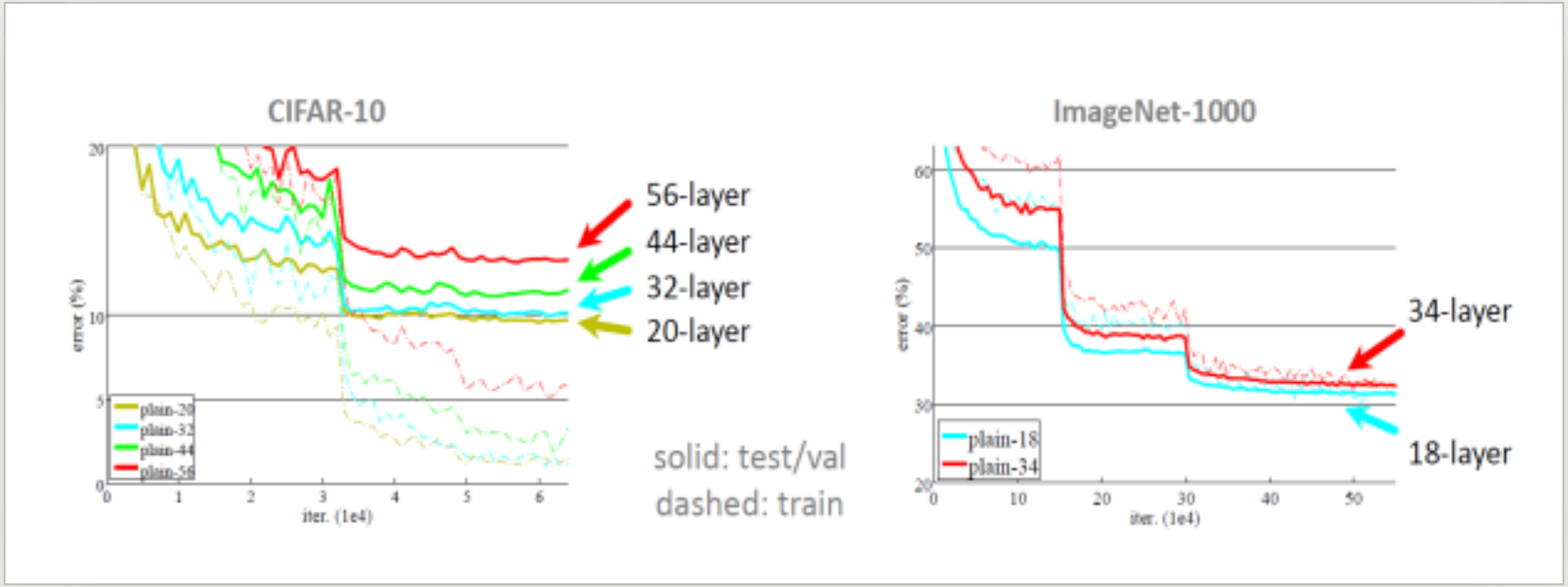
Deeper VGG: 56-layer plain network

- Plain nets: stacking 3x3 conv layers
- - 56-layer net has higher training error and test error than 20-layers net



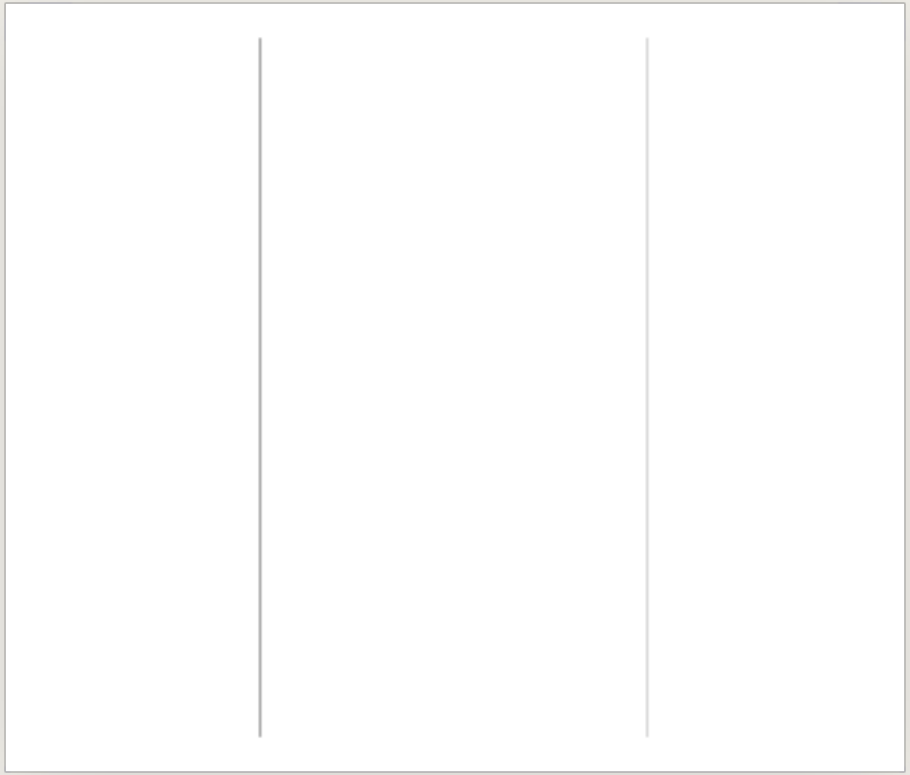
Deeper VGG: overly deep plain nets

- “Overly deep” plain nets have higher training error
- A general phenomenon, observed in many datasets



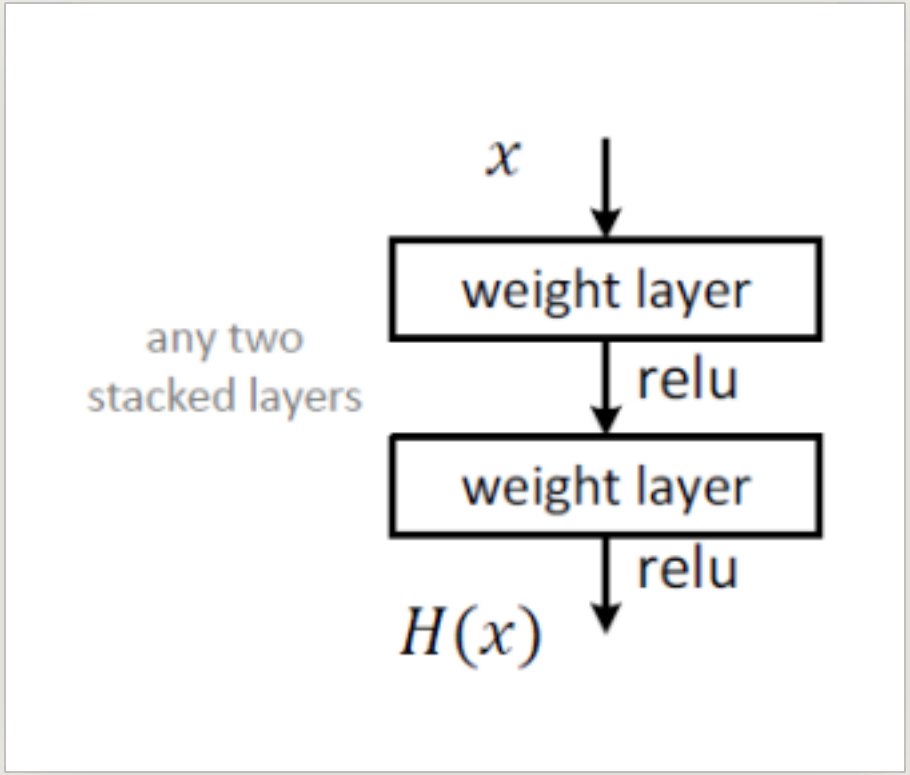
Deeper VGG: optimal mappings

- Deeper networks challenges
  - Features in same level will be almost same
  - An amount of changes is fixed
  - Adding layers make smaller
  - differences
  - Optimal mappings closer to an identity



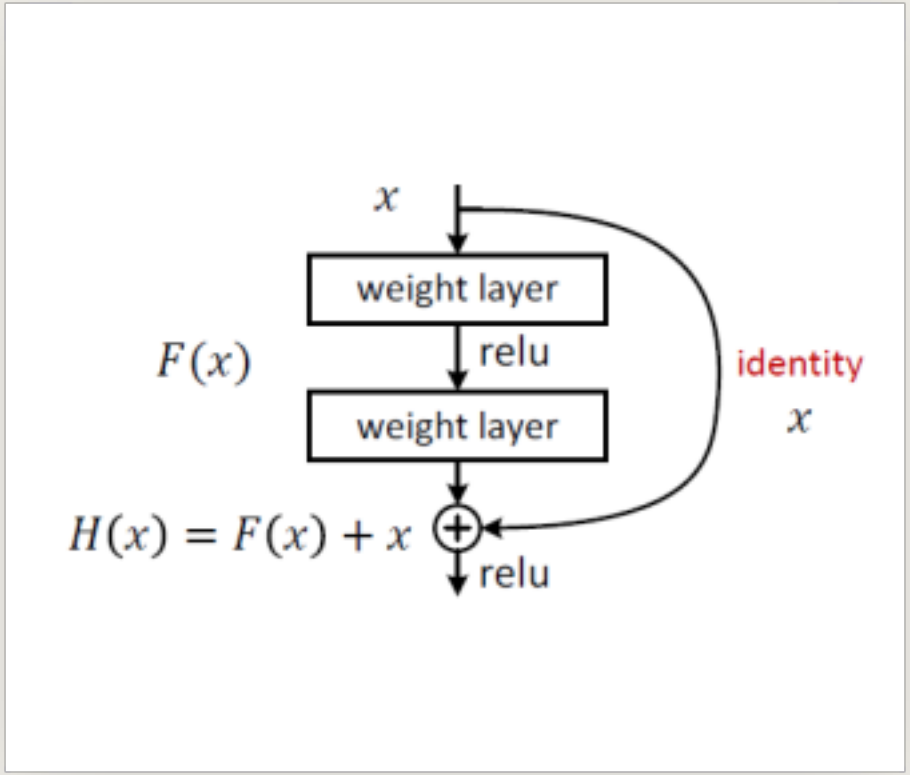
Residual network: plain block

- Plain block:
  - Difficult to make identity mapping because of multiple non-linear layers



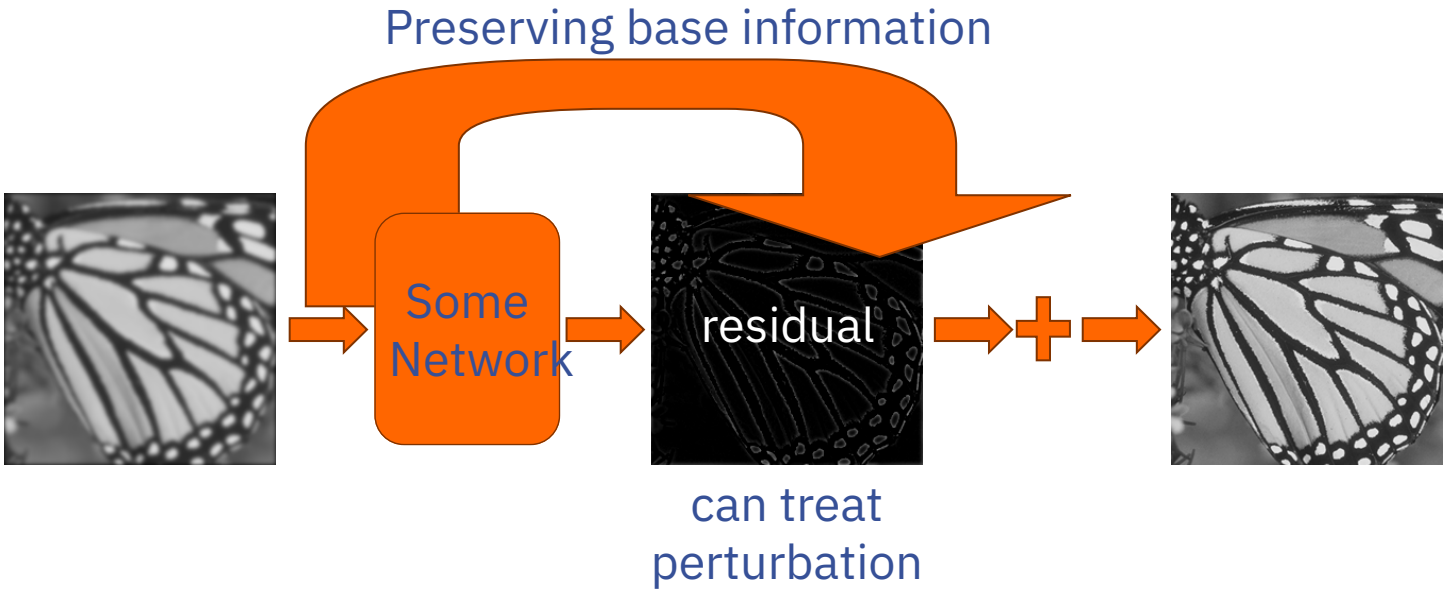
Residual network: residual block

- Residual block:
  - If identity were optimal, easy to set weights as 0
  - If optimal mapping is closer to identity, easier to find small fluctuations
  - -> Appropriate for treating perturbation as keeping a base information



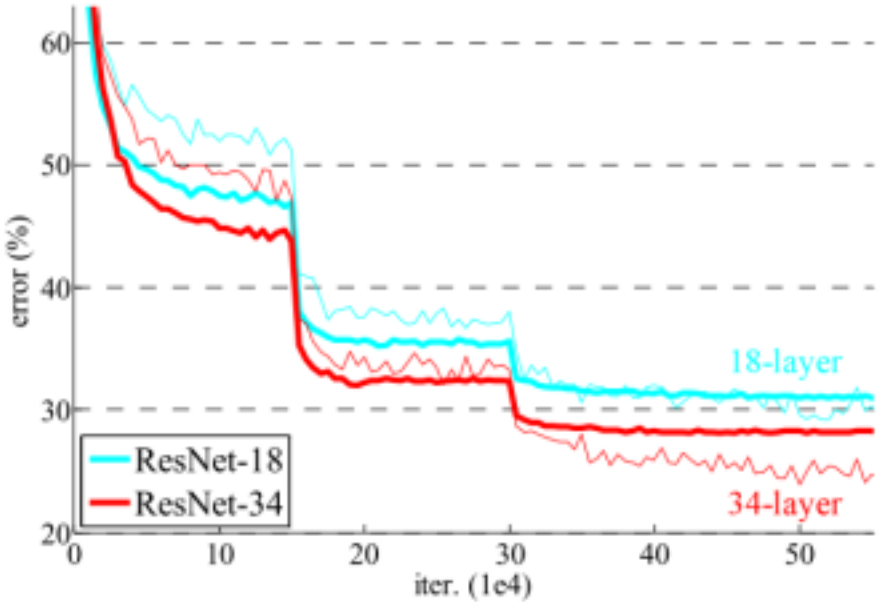
Residual network: preserving base information

- Difference between an original image and a changed image



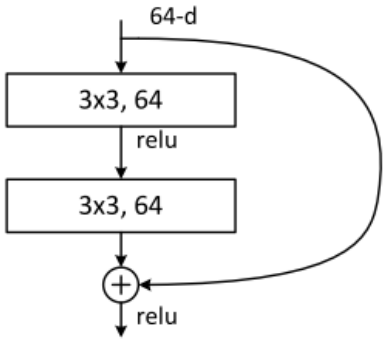
Residual network: training error

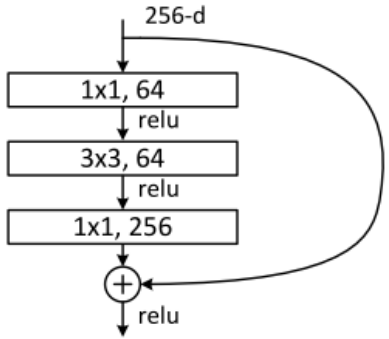
- Deeper ResNets have lower training error



Residual network: residual blocks

- Residual block
  - Very simple
  - Parameter-free





A naïve residual block “bottleneck” residual block
(for ResNet-50/101/152)

Residual network: shortcut connections

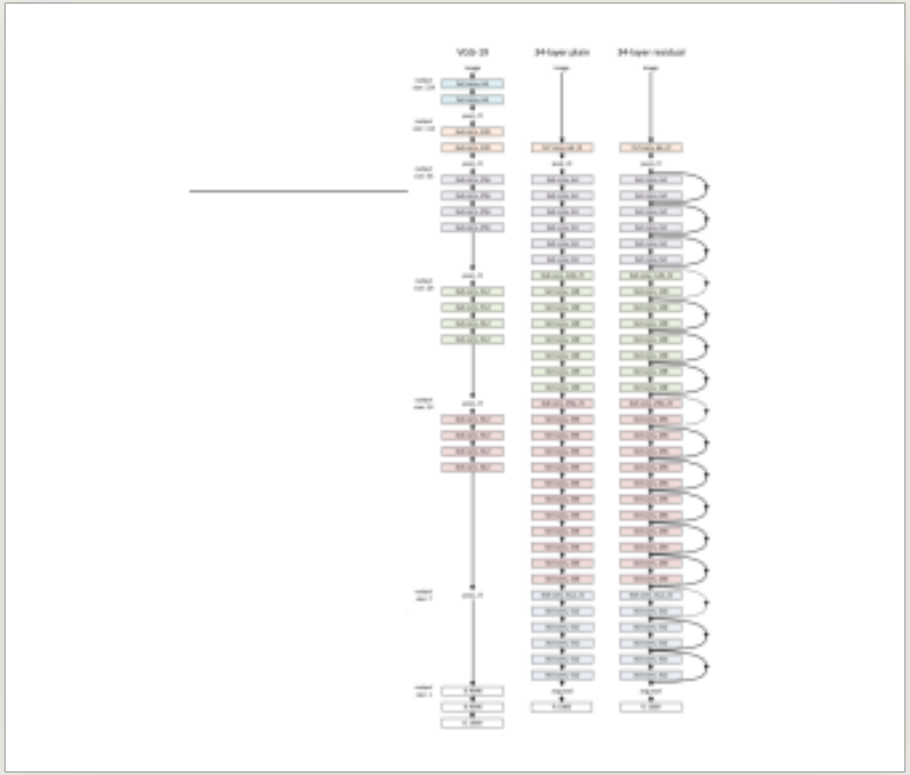
- Shortcuts connections
  - Identity shortcuts
  - Projection shortcuts

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}.$$

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x}.$$

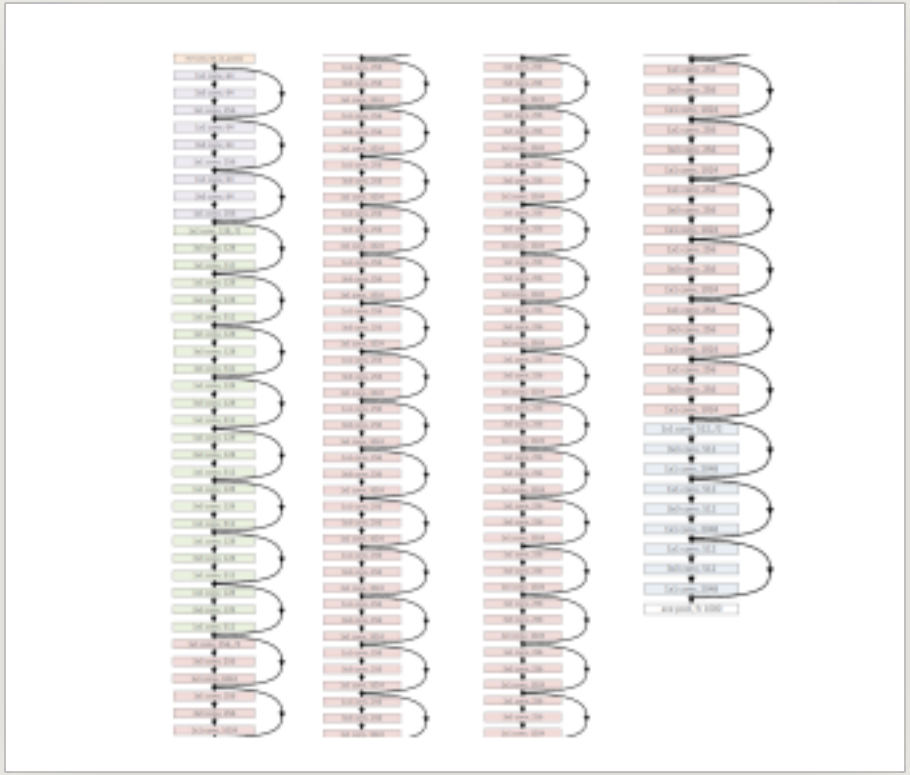
Network design

- Basic design (VGG-style)
  - All 3x3 conv (almost)
  - Spatial size/2 => #filters x2
  - Batch normalization
  - Simple design, just deep
- Other remarks
  - No max pooling (almost)
  - No hidden fc
  - No dropout



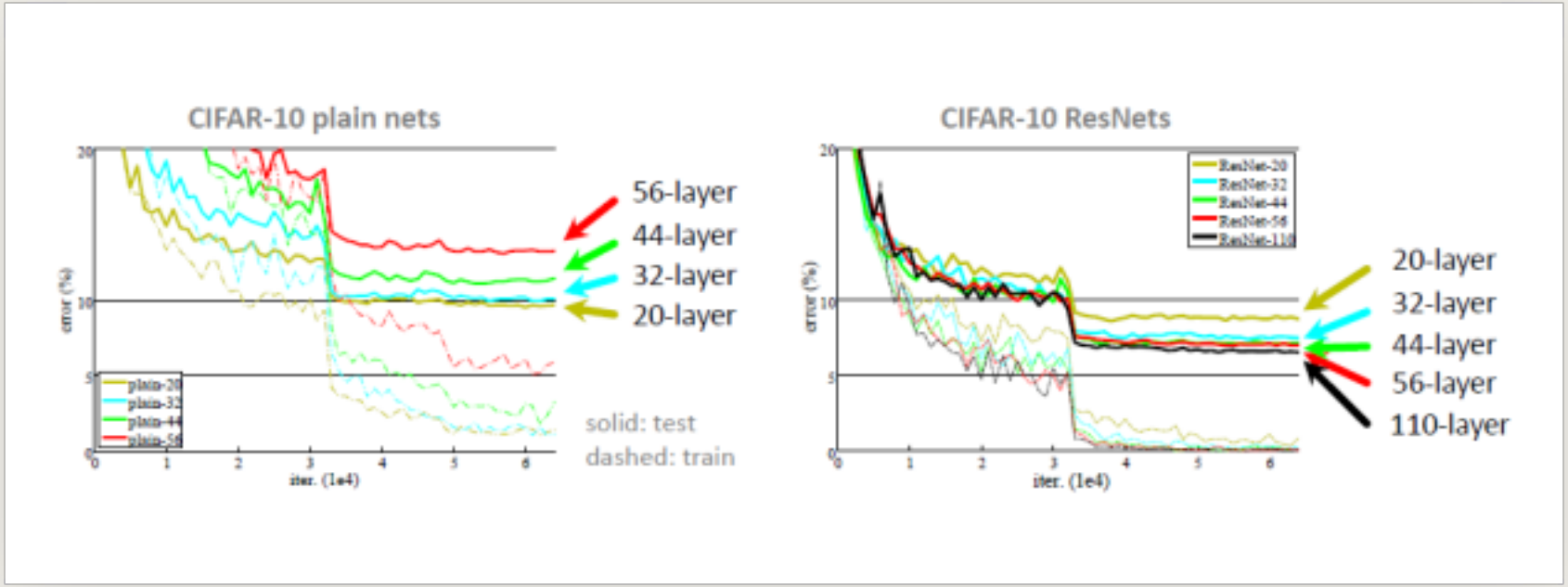
Network design: ResNet-152

- ResNet-152
  - Use bottlenecks
  - ResNet-152 (11.3 billion FLOPs) lower complexity than VGG-16/19 nets (15.3/19.6 billion FLOPs)
- About 64M parameters



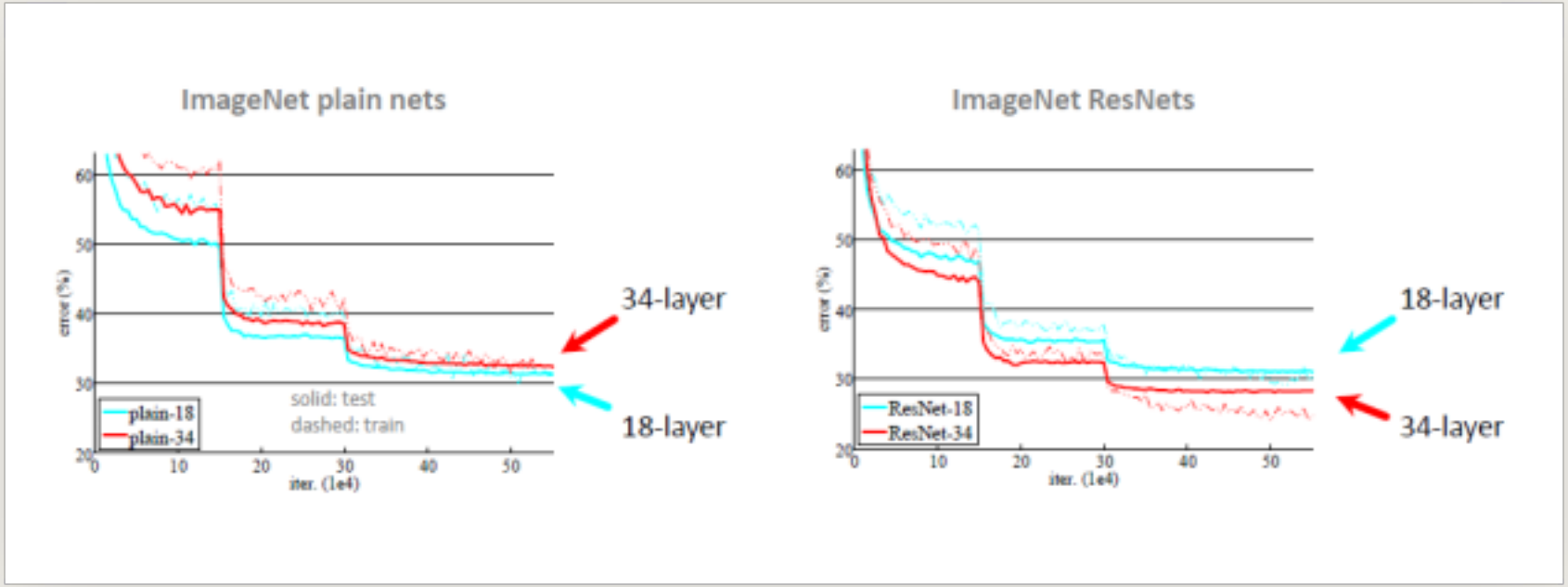
Results

- Deep Resnets can be trained without difficulties
- Deeper ResNets have lower training error, and also lower test error



Results

- Deep Resnets can be trained “without difficulties”
- Deeper ResNets have lower training error, and also lower test error

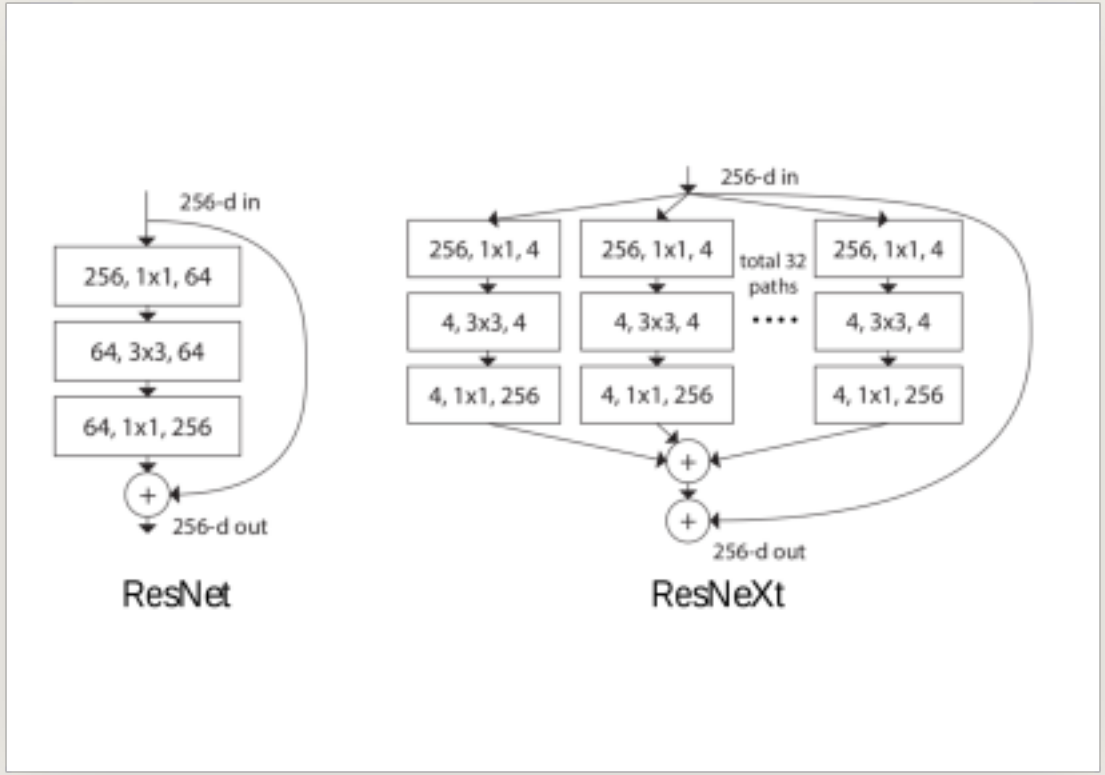


Results: ILSVRC & COCO 2015

- 1st places in all five main tracks in “ILSVRC & COCO 2015 Competitions”
  - ImageNet Classification
  - ImageNet Detection
  - ImageNet Localization
  - COCO Detection
  - COCO Segmentation

Deep ConvNets for image classification

- ResNeXt
  - Multi-branch architecture



Conclusion

- ResNet: currently the best ConvNet framework for large scale image classification
- Fully Convolutional Net (FCN) very interesting option
- Limitation: at each layer, convolution only considers local interaction/attention (limited by the size of the filter)

PART 2 / 3

Vision transformers

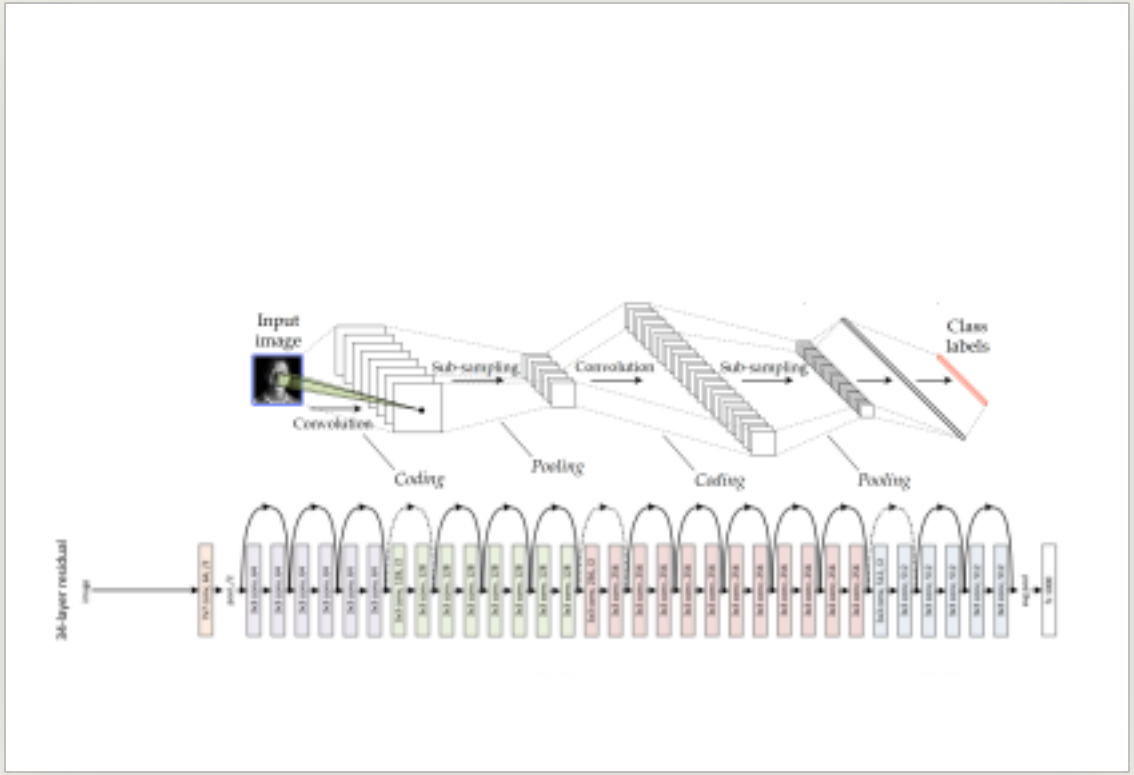
01 Large ConvNets

02 **Vision transformers**

03 Deep residual learning

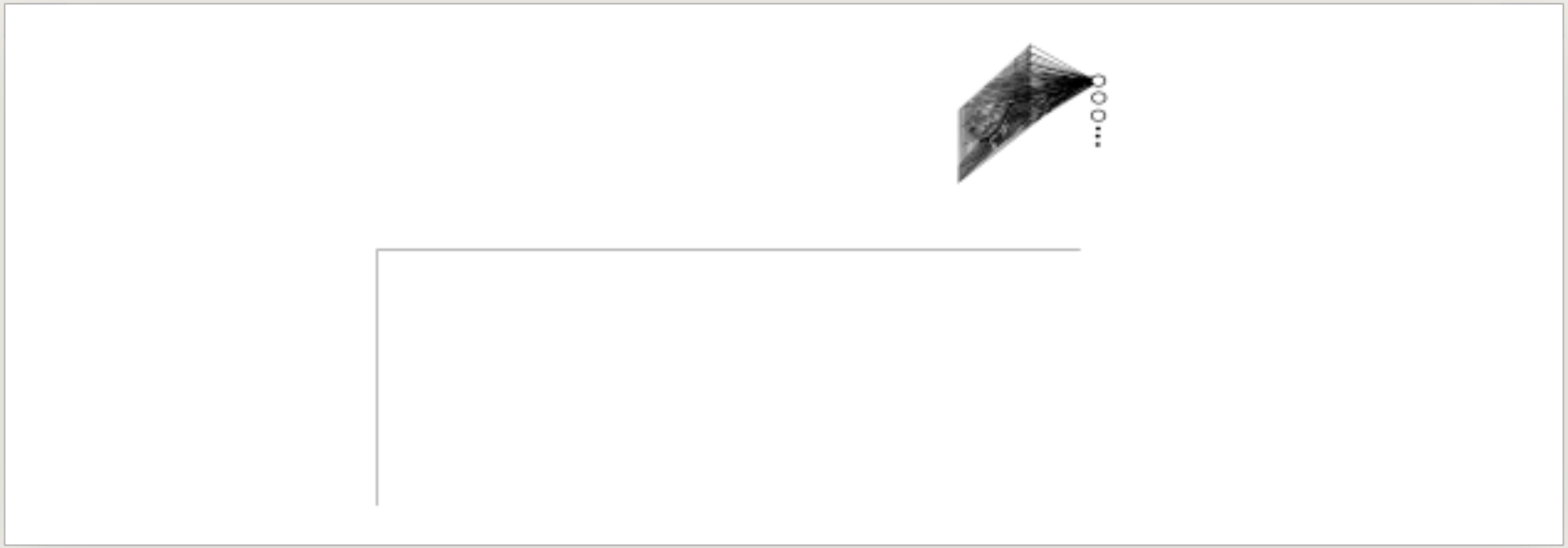
Image classification: where we are and what is missing?

- The 2000s: *BoWs + SVM*
- The 2010s: Very *Large* ConvNets
- 2020: The star: **ResNet**
- **Next step?**
- **What is missing?**



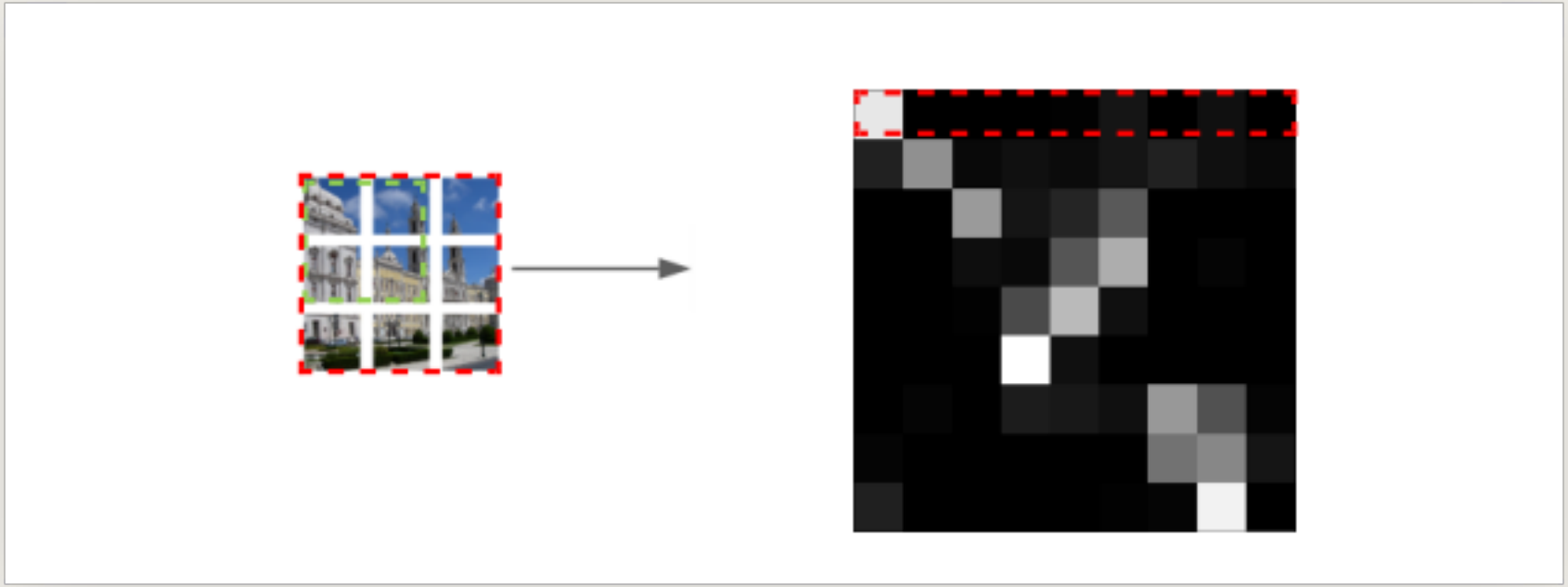
Attention process in ConvNets

- In ConvNets, what information is shared between pixels (or features) in one block? => *2D spatial locality (typically 3x3) => attention is done locally*
- *Rq: less local after many layers*



Global (self) attention

- How to build a deep architecture with local global attention inside? Meaning that one patch may interact with all others!
- => Different than convNet!



PART 2 · VISION TRANSFORMERS

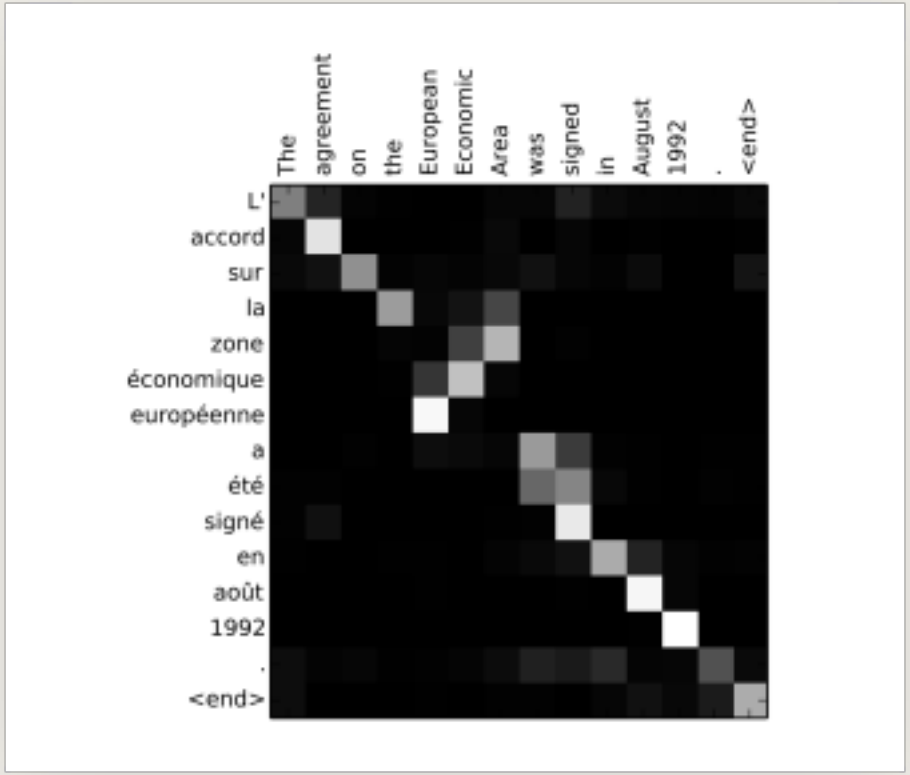
NLP: Attention is all you need

01 **NLP: Attention is all you need**

02 Transformer for image classification

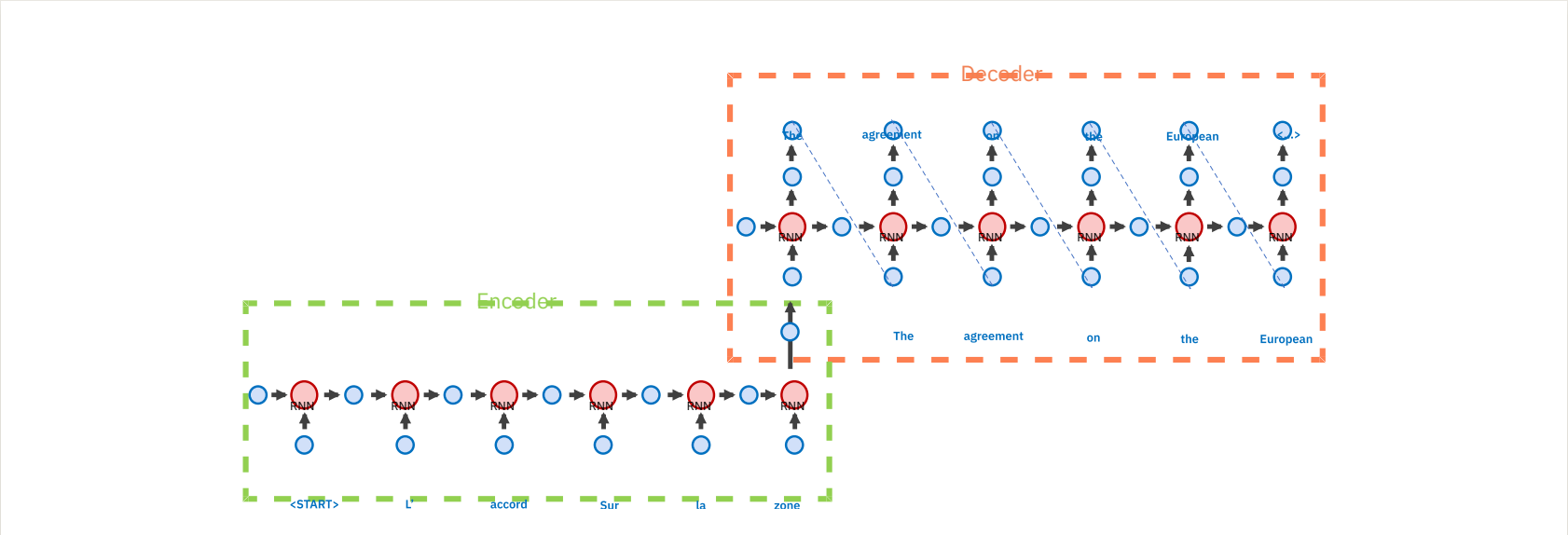
What do they do in natural language processing?

- Attention between words in Machine translation process:
-
- Use them to compute new features



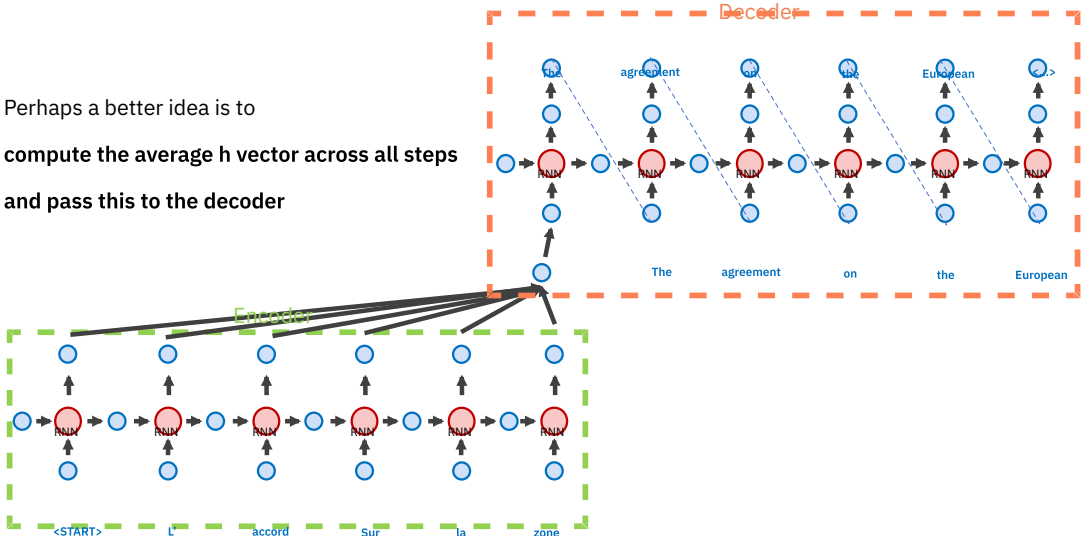
Attention process in NLP

- Basic language translation models: Encoder/Decoder
- Ex.: Seq2Seq -- RNNs2RNNs
- Cross-attention for language translation in at the end of Encoder



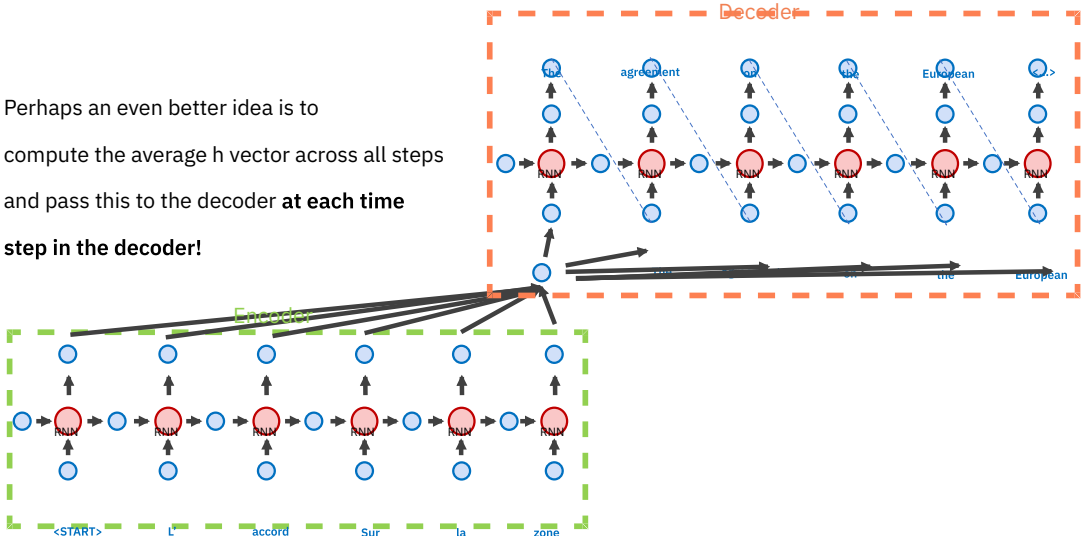
Attention process in NLP

- Basic language translation models: Encoder/Decoder
- Cross-attention:



Attention process in NLP

- Basic language translation models: Encoder/Decoder
- Cross-attention:



Attention process in NLP

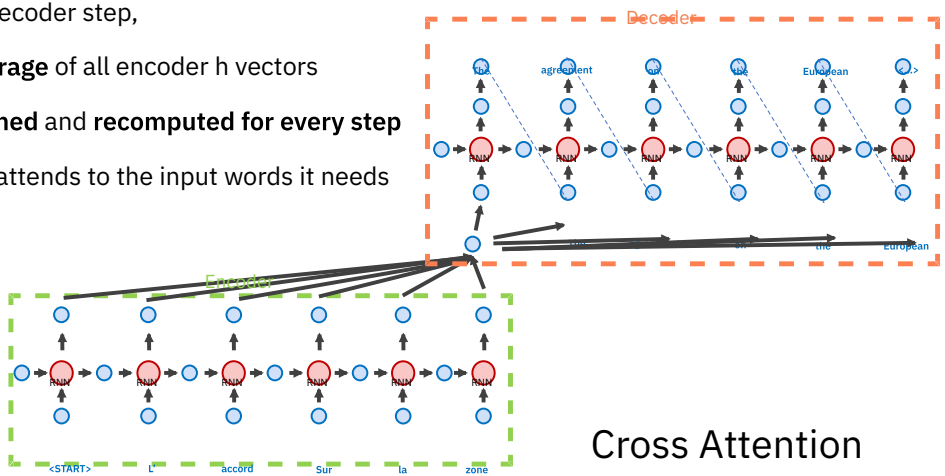
- Basic language translation models: Encoder/Decoder
- Cross-attention:

Better still: at each decoder step,

pass a **weighted average** of all encoder h vectors

The weights are **learned** and **recomputed for every step**

⇒ each output word attends to the input words it needs

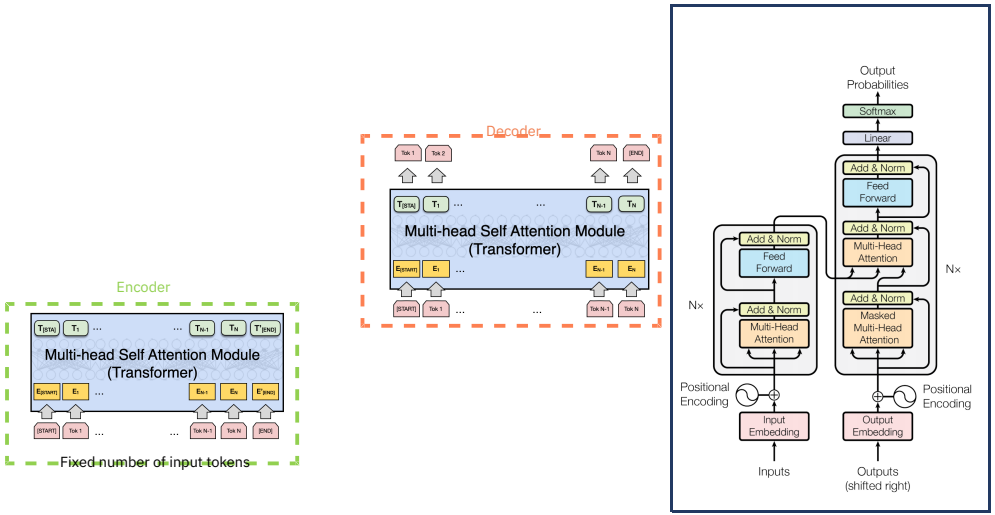


Cross Attention

Encoder/ Decoder

The Transformer architecture

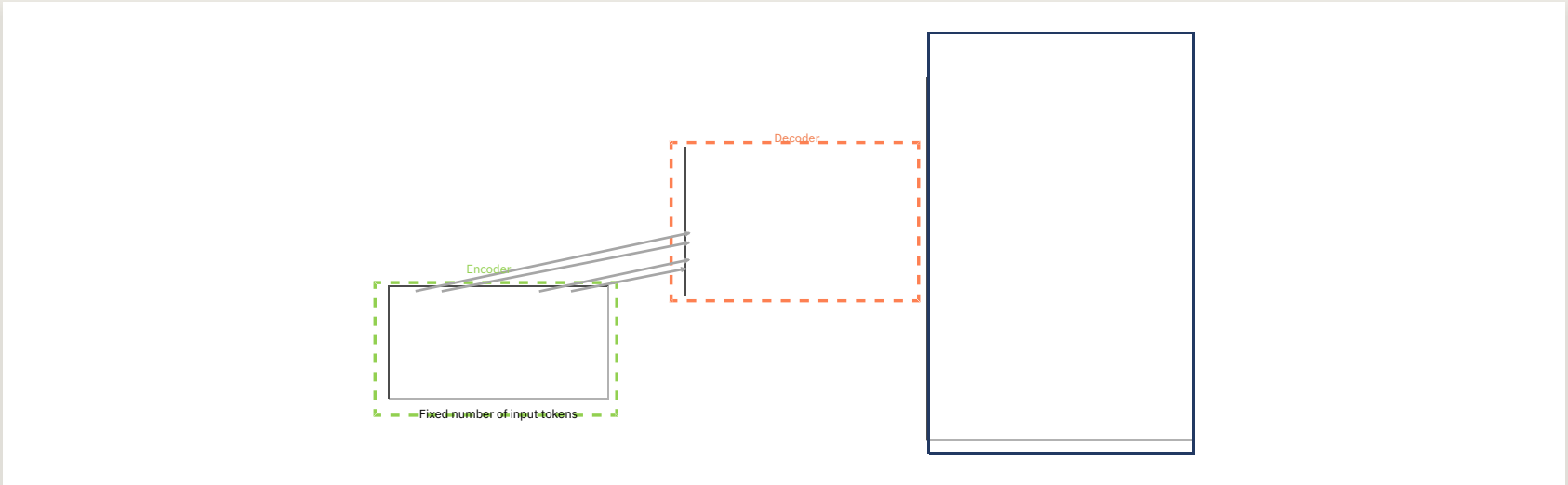
- Basic language translation models: Encoder/Decoder
- **Transformer** architecture (no RNNs)



Vaswani et al. Attention is all you need. NeurIPS 2017 · arxiv.org/abs/1706.03762

The Transformer architecture

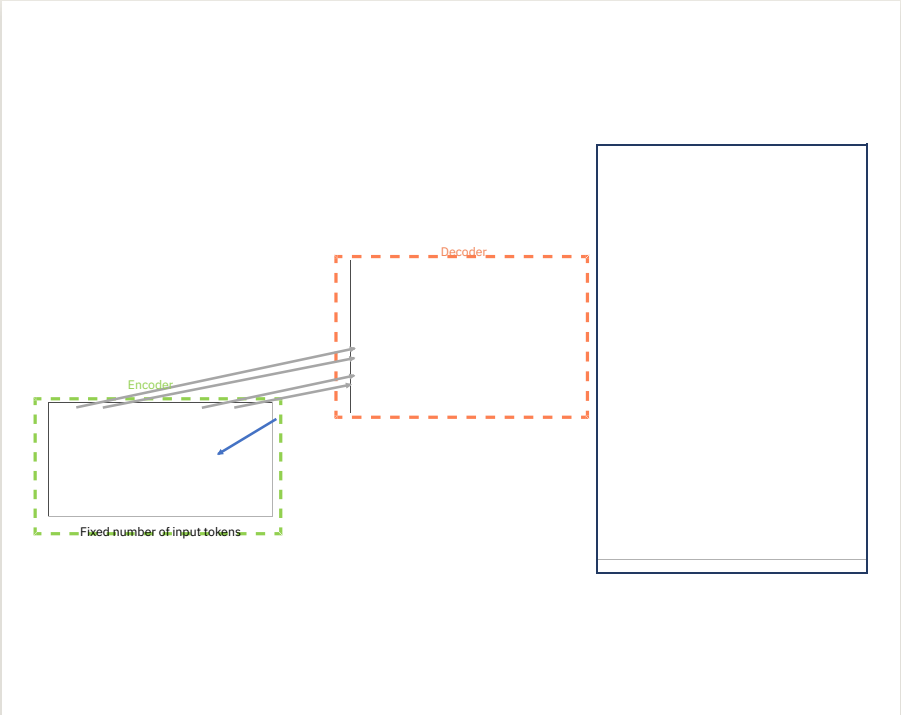
- Basic language translation models: Encoder/Decoder
- **Transformer** architecture (no RNNs)
- Cross-attention



Vaswani et al. Attention is all you need. NeurIPS 2017 · arxiv.org/abs/1706.03762

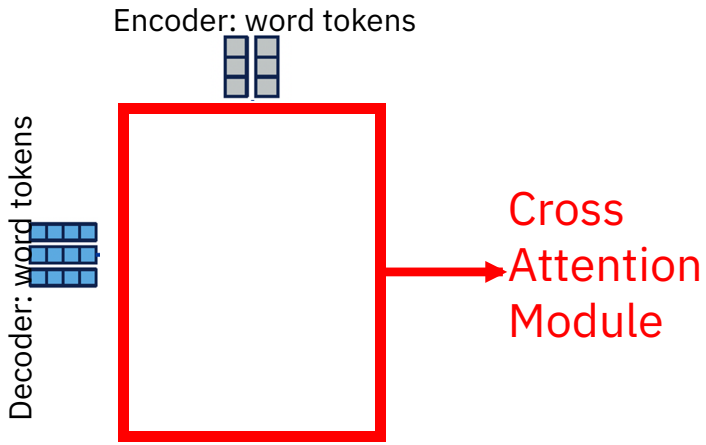
The Transformer architecture

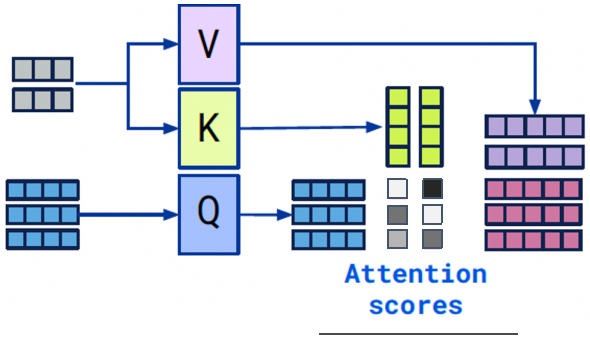
- Basic language translation models: Encoder/ Decoder
- **Transformer** architecture (no RNNs)
- Cross-attention
- Self-attention



Vaswani et al. Attention is all you need. NeurIPS 2017 · arxiv.org/abs/1706.03762

Cross-attention module





$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

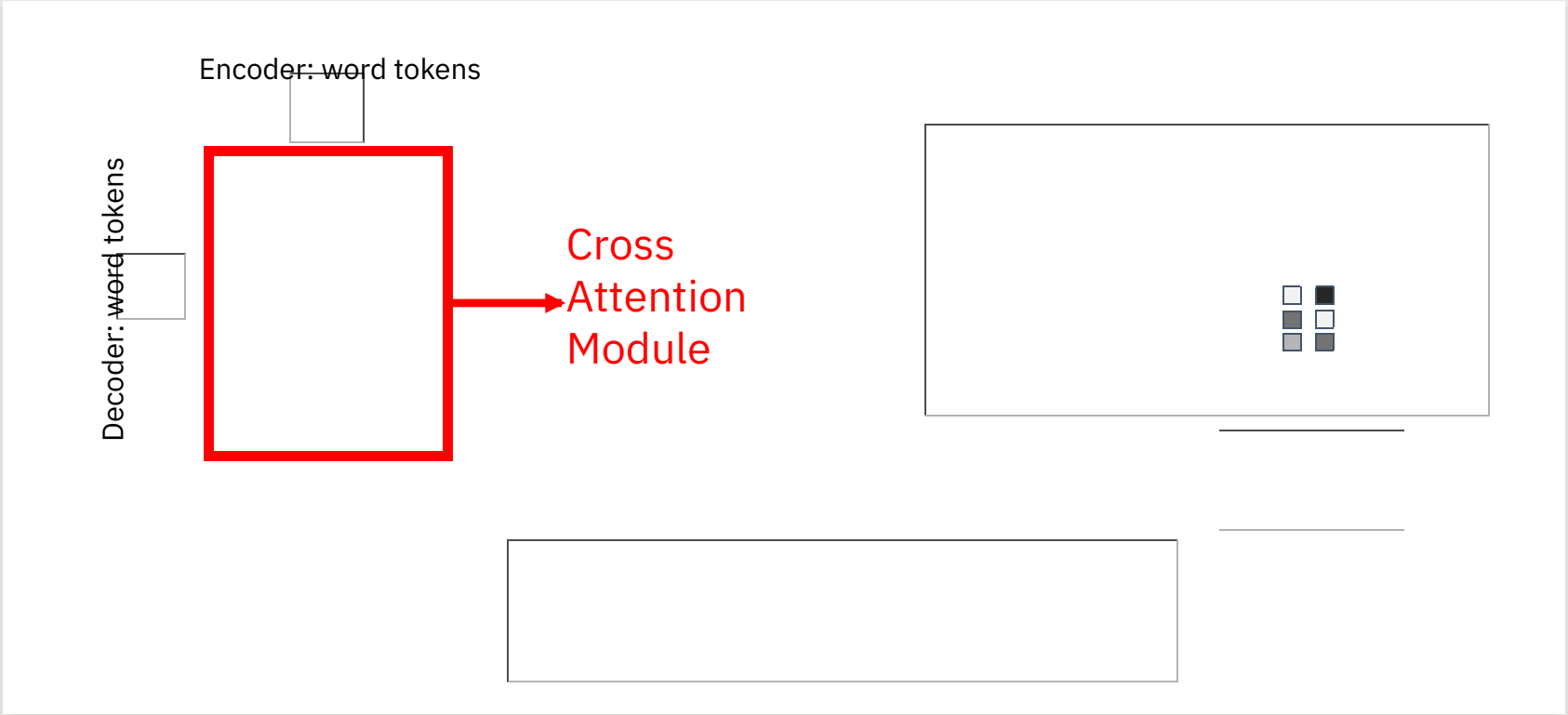
PART 2 · VISION TRANSFORMERS

Transformer for image classification

01 NLP: Attention is all you need

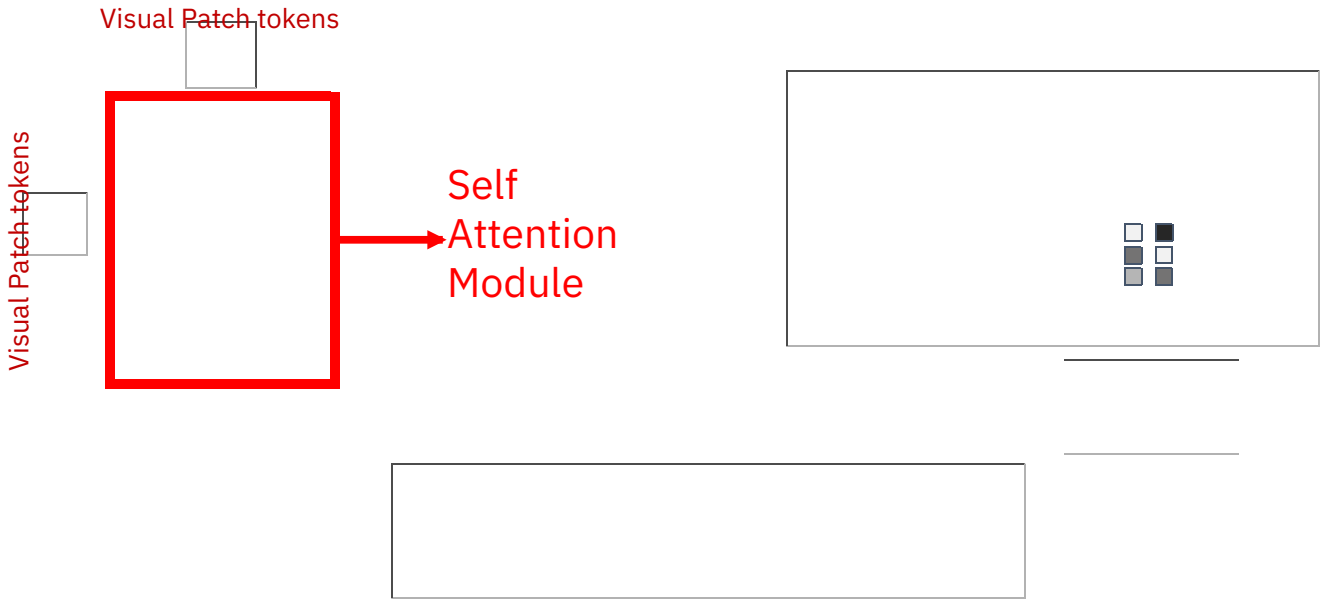
02 **Transformer for image classification**

Cross-attention module



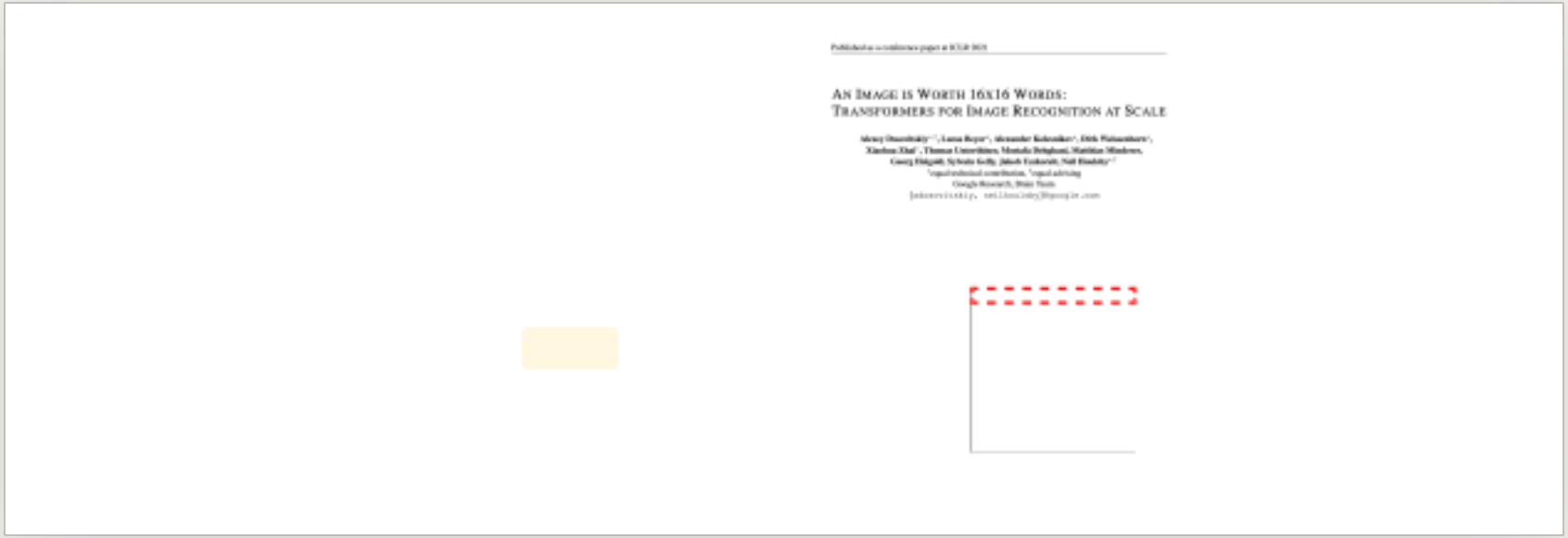
Self-attention module

- Very similar except that Visual token is definitively less natural than word for NLP



Attention process in vision

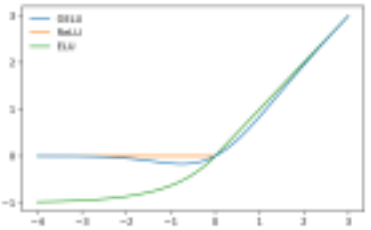
- Is it possible to mimic this attention-based architecture for vision processing?
- Yes! **ViT** (Vision image Transformers) architecture
- => Self attention encoder modules for classification



ViT: Vision Transformer architecture

- [class=CLS] token: a learnable embedding to the sequence of embedded patches
- Layernorm (LN) before every block, and residual connections after every block
- MSA: Multi Head Self Attention
- MLP: two layers with a GELU non-linearity





$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1\mathbf{E}; \mathbf{x}_p^2\mathbf{E}; \cdots; \mathbf{x}_p^N\mathbf{E}] + \mathbf{E}_{\text{pos}}, \qquad \mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D},\ \mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$$

$$\mathbf{z}'_\ell = \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, \qquad \ell = 1 \ldots L$$

$$\mathbf{z}_\ell = \text{MLP}(\text{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell, \qquad \ell = 1 \ldots L$$

$$\mathbf{y} = \text{LN}(\mathbf{z}_L^0)$$

Layer normalization (LN)

- Normalizes **each token on its own**, over its d features
- Independent of the batch: same computation at training and test time
- Learned scale γ and shift β per feature
- In ViT: before every MSA and MLP block (**pre-norm**)

FOR ONE TOKEN $x \in \mathbb{R}^D$

$$\mu = \frac{1}{d} \sum_i x_i$$

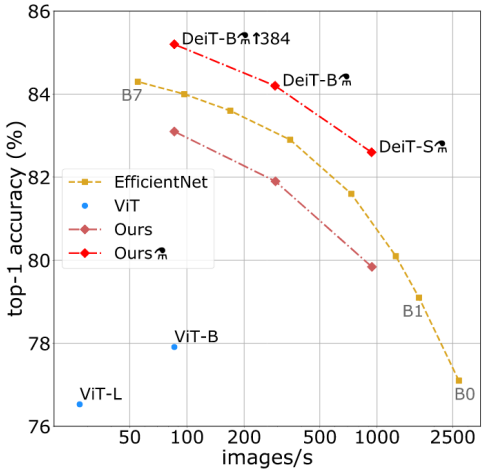
$$\sigma^2 = \frac{1}{d} \sum_i (x_i - \mu)^2$$

$$\mathrm{LN}(x) = \gamma \odot (x - \mu) / \sqrt{(\sigma^2 + \varepsilon)} + \beta$$

	BatchNorm	LayerNorm
Statistics	per feature, over the batch	per token, over its features
Used in	ConvNets (ResNet)	Transformers (ViT)

Experiments with ViT, DeiT and CaiT

- Experiments with ViT (and variants DeiT, CaiT) transformers for image classification
- State-of-the-art performance on ImageNet1k classification!
- From ViT paper, **many tricks/discussions to simplify learning** in DeiT, CaiT, ...



Published as a conference paper at ICML 2021

DeiT

Training data-efficient image transformers & distillation through attention

Hugo Touvron [1 2] Matthieu Cord [1 2] Matthijs Douze [1]
Francisco Massa [1] Alexandre Sablayrolles [1] Hervé Jégou [1]

PART 3 / 3

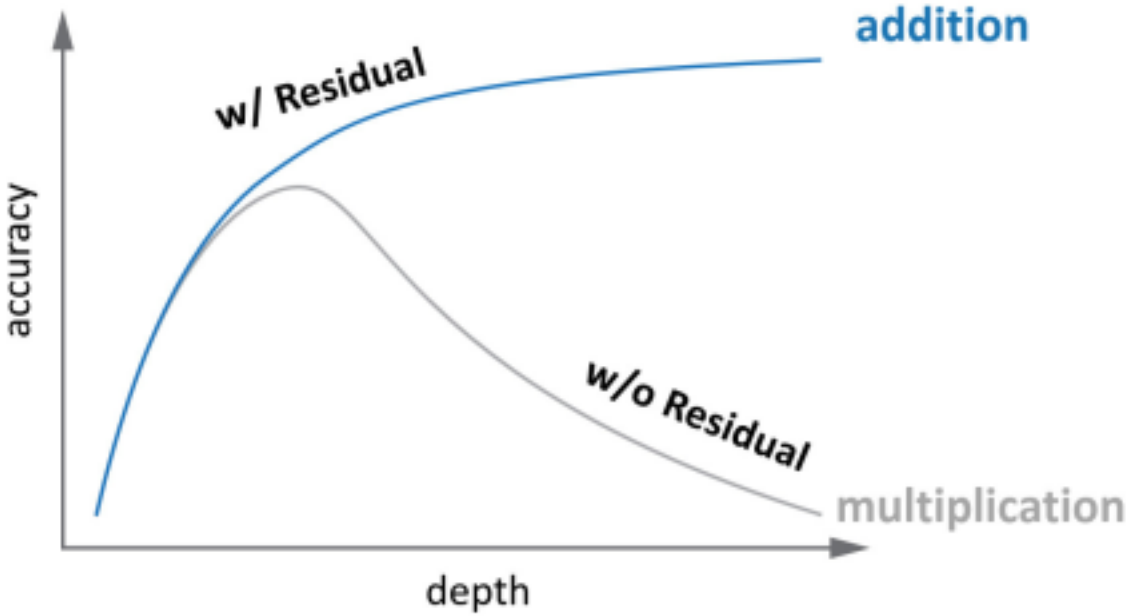
Deep residual learning

01 Large ConvNets

02 Vision transformers

03 **Deep residual learning**

Understanding residual learning



Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. "Deep Residual Learning for Image Recognition". arXiv 2015, CVPR 2016.

Understanding residual learning

Vanilla network

$$x_{l+1} = W_l x_l$$

recursively

$$x_L = \left(\prod W_i\right) x_l$$

gradients

$$\frac{\partial \mathcal{E}}{\partial x_l} = \frac{\partial \mathcal{E}}{\partial x_L}\left(\prod W_i\right)$$

Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. "Identity Mappings in Deep Residual Networks", ECCV 2016

Understanding residual learning

Vanilla network

$$x_{l+1} = W_l x_l$$

recursively

$$x_L = \left(\prod W_i\right) x_l$$

gradients

$$\frac{\partial \mathcal{E}}{\partial x_l} = \frac{\partial \mathcal{E}}{\partial x_L}\left(\prod W_i\right)$$

- **vanishing signal**

Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. "Identity Mappings in Deep Residual Networks", ECCV 2016

Understanding residual learning

Residual network

$$x_{l+1} = x_l + F_l(x_l)$$

recursively

$$x_L = x_l + \sum F_i(x_i)$$

gradients

$$\frac{\partial \mathcal{E}}{\partial x_l} = \frac{\partial \mathcal{E}}{\partial x_L}\left(1 + \frac{\partial}{\partial x_l}\sum F_i(x_i)\right)$$

Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. "Identity Mappings in Deep Residual Networks", ECCV 2016

Understanding residual learning

Residual network

$$x_{l+1} = x_l + F_l(x_l)$$

recursively

$$x_L = x_l + \sum F_i(x_i)$$

gradients

$$\frac{\partial \mathcal{E}}{\partial x_l} = \frac{\partial \mathcal{E}}{\partial x_L}\left(1 + \frac{\partial}{\partial x_l}\sum F_i(x_i)\right)$$

- not vanishing

Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. "Identity Mappings in Deep Residual Networks", ECCV 2016

Understanding residual learning

Residual network

$$x_{l+1} = x_l + F_l(x_l)$$

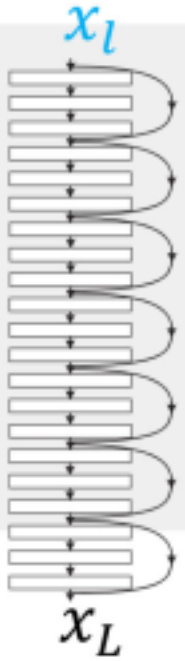
recursively

$$x_L = x_l + \sum F_i(x_i)$$

gradients

$$\frac{\partial \mathcal{E}}{\partial x_l} = \frac{\partial \mathcal{E}}{\partial x_L}\left(1 + \frac{\partial}{\partial x_l}\sum F_i(x_i)\right)$$

- not vanishing



Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. "Identity Mappings in Deep Residual Networks", ECCV 2016

Understanding residual learning

Residual network

$$x_{l+1} = x_l + F_l(x_l)$$

recursively

$$x_L = x_l + \sum F_i(x_i)$$

gradients

$$\frac{\partial \mathcal{E}}{\partial x_l} = \frac{\partial \mathcal{E}}{\partial x_L}\left(1 + \frac{\partial}{\partial x_l}\sum F_i(x_i)\right)$$

- **not vanishing**



Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. "Identity Mappings in Deep Residual Networks", ECCV 2016

Understanding residual learning

Residual network

$$x_{l+1} = x_l + F_l(x_l)$$

recursively

$$x_L = x_l + \sum F_i(x_i)$$

gradients

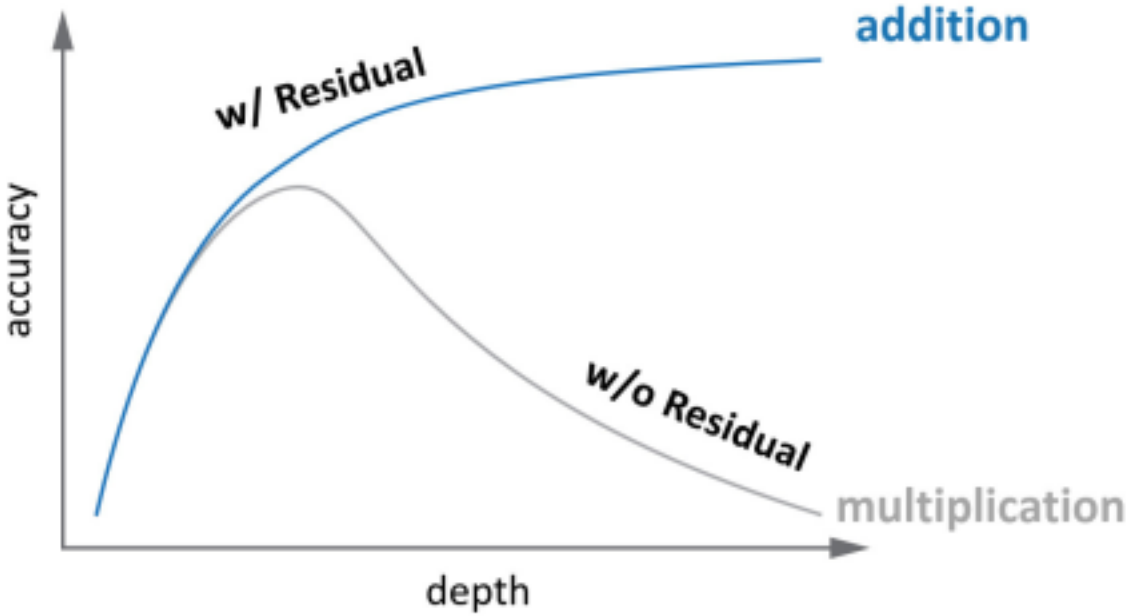
$$\frac{\partial \mathcal{E}}{\partial x_l} = \frac{\partial \mathcal{E}}{\partial x_L}\left(1 + \frac{\partial}{\partial x_l}\sum F_i(x_i)\right)$$

- not vanishing



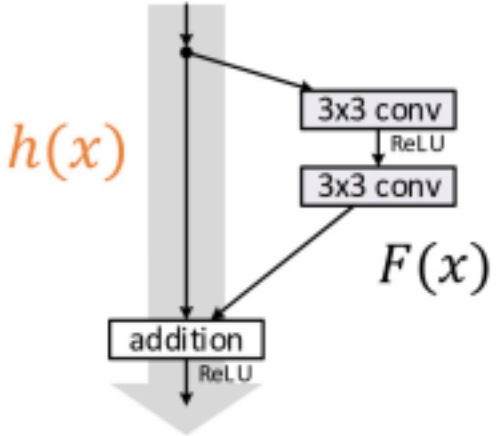
Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. "Identity Mappings in Deep Residual Networks", ECCV 2016

Understanding residual learning



Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. "Deep Residual Learning for Image Recognition". arXiv 2015, CVPR 2016.

Identity mappings in residual learning



- Residual Learning: h = identity mapping

Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. "Identity Mappings in Deep Residual Networks", ECCV 2016

If *h* is multiplicative ...

consider $h(x) = \lambda x$:

$$x_{l+1} = \lambda x_l + F_l(x_l)$$

recursively

$$x_L = \lambda^{L-l} x_l + \sum \hat{F}_i(x_i)$$

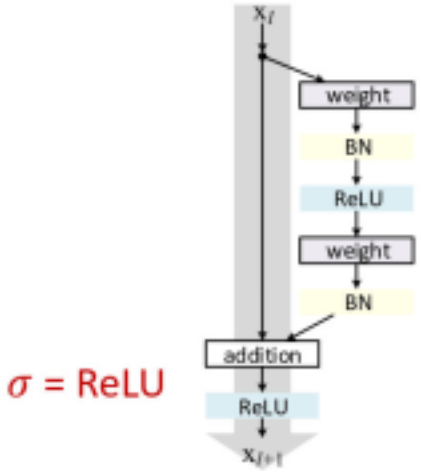
gradients

$$\frac{\partial \mathcal{E}}{\partial x_l} = \frac{\partial \mathcal{E}}{\partial x_L}\left(\lambda^{L-l} + \frac{\partial}{\partial x_l}\sum \hat{F}_i(x_i)\right)$$

- **vanishing signals, if $\lambda < 1$**

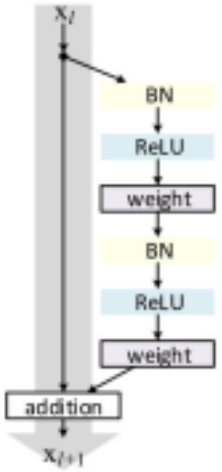
Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. "Identity Mappings in Deep Residual Networks", ECCV 2016

Pre-activation (pre-normalization)



σ = ReLU

post-activation

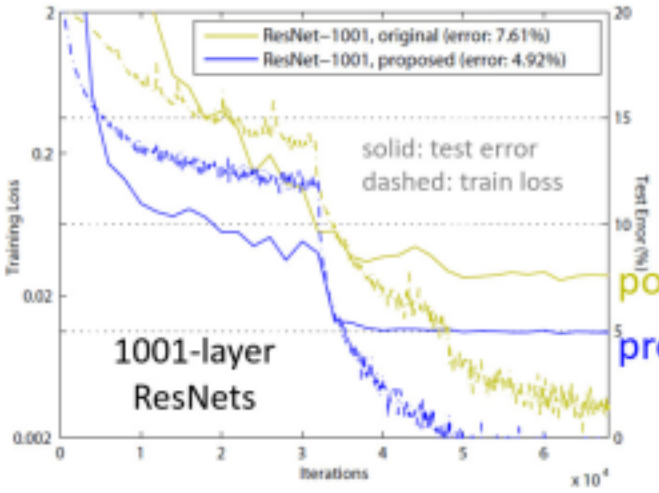


σ = identity

pre-activation

Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. "Identity Mappings in Deep Residual Networks", ECCV 2016

1001-layer ResNets



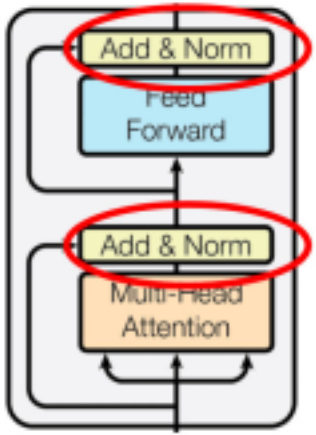
post-activation (σ = ReLU)

pre-activation (σ = identity)

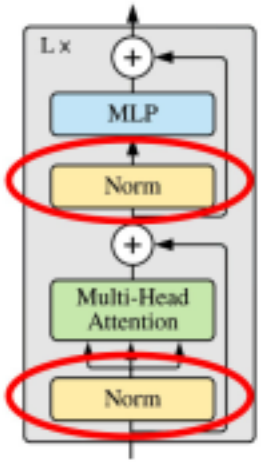
- ReLU can also block signals when there are many layers
- Pre-activation eases optimization

Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. "Identity Mappings in Deep Residual Networks", ECCV 2016

Pre-activation (pre-normalization)



post-norm
(original Transformer)



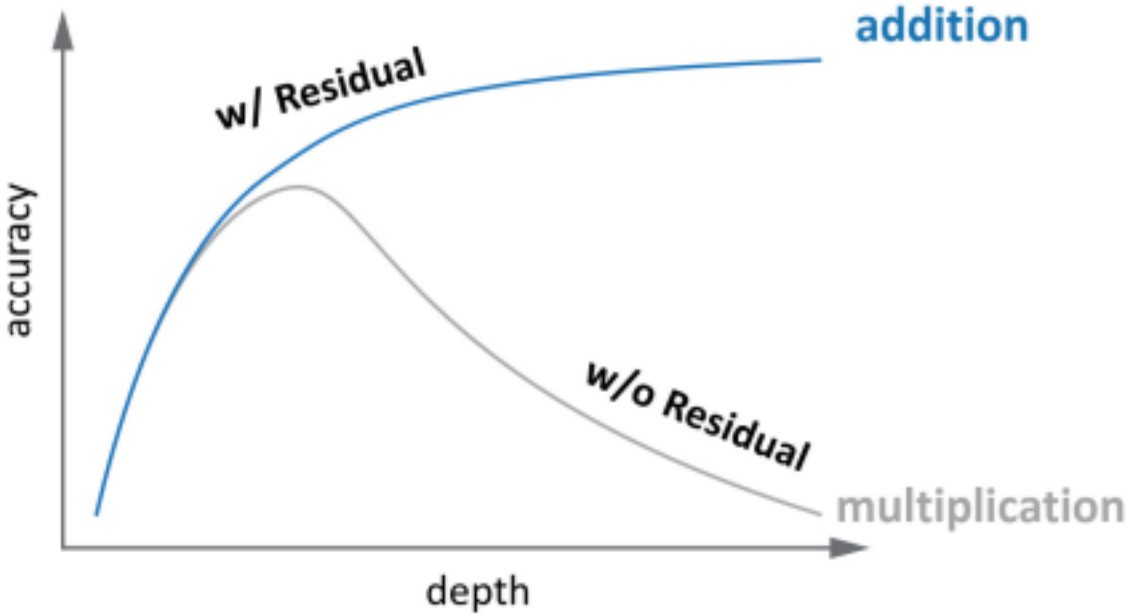
pre-norm
(after GPT-2)

few modifications. Layer normalization (Ba et al., 2016) was moved to the input of each sub-block, similar to a pre-activation residual network (He et al., 2016) and an additional layer normalization was added after the final self-attention block. A modified initialization which accounts

from the GPT-2 paper
[Radford, et al., 2019]

"Attention is all you need", Vaswani, et al., 2017
"Language Models are Unsupervised Multitask Learners" (GPT-2), Radford, et al., 2019

Understanding residual learning



Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. "Deep Residual Learning for Image Recognition". arXiv 2015, CVPR 2016.

Understanding residual learning

The Shattered Gradients Problem: If resnets are the answer, then what is the question?

David Balduzzi [1] Marcus Frean [1] Lennox Leary [1] JP Lewis [1 2] Kurt Wan-Duo Ma [1] Brian McWilliams [1]

Residual Networks Behave Like Ensembles of Relatively Shallow Networks

Andreas Veit Michael Wilber Serge Belongie
Department of Computer Science & Cornell Tech
Cornell University
{av443, mjw285, sjb344}@cornell.edu

Visualizing the Loss Landscape of Neural Nets

Hao Li[1], Zheng Xu[1], Gavin Taylor[2], Christoph Studer[3], Tom Goldstein[1]
[1]University of Maryland, College Park [2]United States Naval Academy [3]Cornell University
{haoli,xuzh,tomg}@cs.umd.edu, taylor@usna.edu, studer@cornell.edu

Identity Matters in Deep Learning

Moritz Hardt [*] Tengyu Ma [†]

July 23, 2018

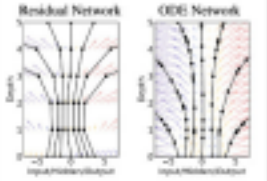
SKIP CONNECTIONS ELIMINATE SINGULARITIES

A. Emin Orhan
eminorhan@gmail.com
Baylor College of Medicine & Rice University

Xaq Pitkow
xaq@rice.edu

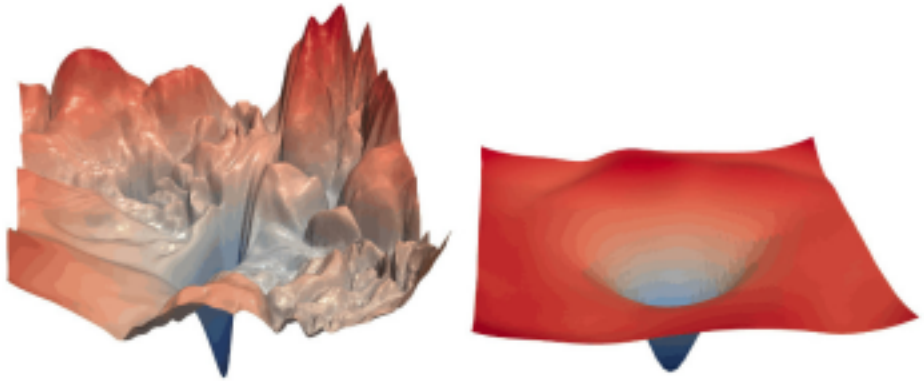
Neural Ordinary Differential Equations

Ricky T. Q. Chen*, Yulia Rubanova*, Jesse Bettencourt*, David Duvenaud
University of Toronto, Vector Institute
{rtqichen, rubanova, jessebett, duvenaud}@cs.toronto.edu



Loss landscapes with residual connections

- Residual connections produce smoother loss landscapes



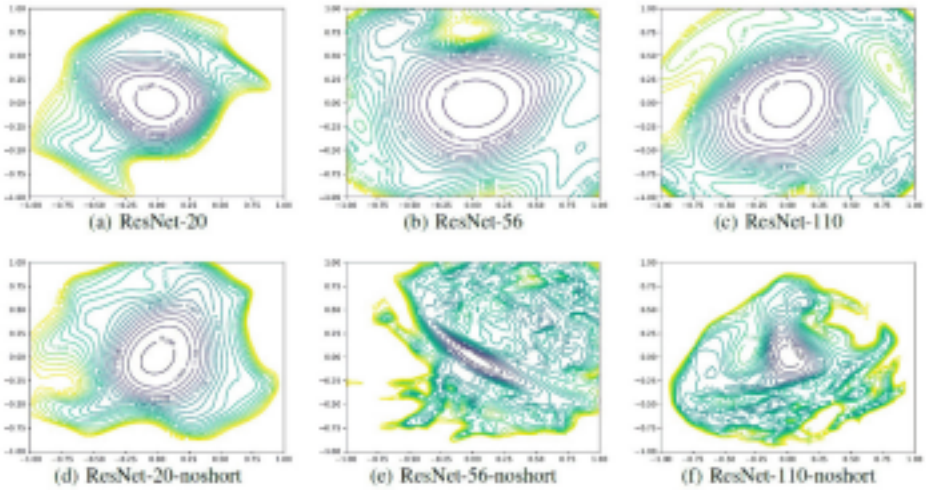
(a) without skip connections (b) with skip connections

Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The vertical axis is logarithmic to show dynamic range. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.

Li, et al. "Visualizing the Loss Landscape of Neural Nets", NeurIPS 2018

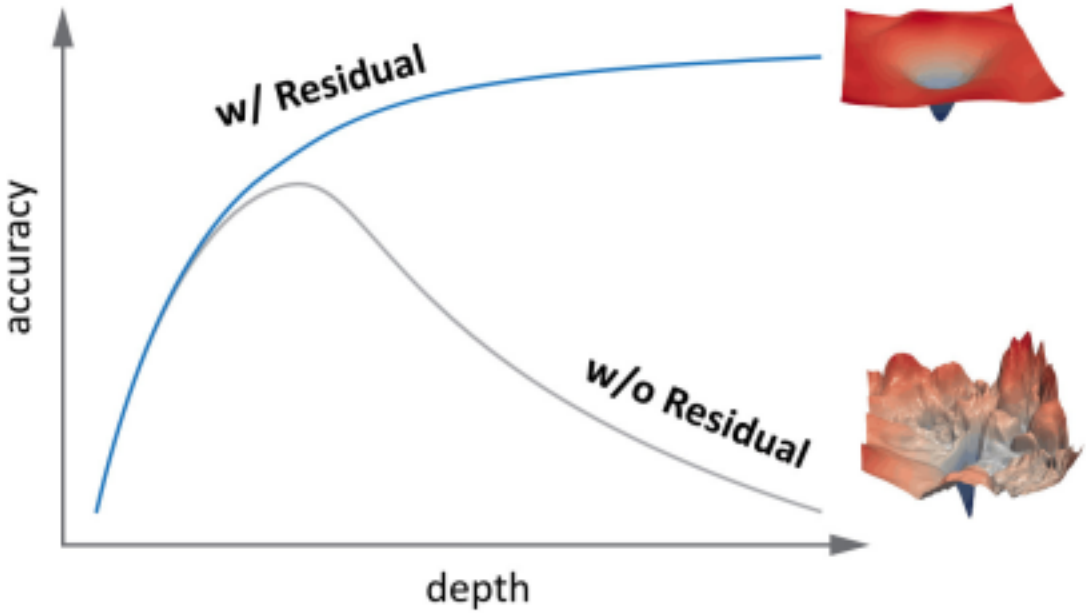
Loss landscapes without residual connections

- Without residual connections, deeper nets have more chaotic landscapes



Li, et al. "Visualizing the Loss Landscape of Neural Nets", NeurIPS 2018

Understanding residual learning



Li, et al. "Visualizing the Loss Landscape of Neural Nets", NeurIPS 2018

THANK YOU

Questions?

Clément Rambour

clement.rambour@sorbonne-universite.fr

clementrambour.github.io/teaching-rdfia